Das einheitliche Shared-Library-Format `.sl` Standardisierte Ladeprozedur

Formale Analyse der Referenzimplementierung `sharedlib` und ihre Wiederverwendung in Gen-DB und Subgraph-SAT-Solver

Stephan Epp

`M.Sc.`

21. Oktober 2026

Kurzfassung

Dieselbe Idee heißt unter Windows `.dll`, unter Linux `.so` und unter macOS `.dylib`. Dahinter stehen drei Binärformate, drei Lader und drei Regeln für Sichtbarkeit, Suchpfade und Fehlermeldungen. Diese Arbeit führt das Kapitel zum Laden der C/C++-Bibliothek aus der Arbeit zur Wiederverwendung des Subgraph Algorithmus (`csubgraph.tex`) fort und entwickelt es zu einer eigenständigen Untersuchung des Vorschlags eines einheitlichen Formats `.sl` mit *einer* standardisierten, betriebssystemunabhängigen Ladeprozedur. Grundlage sind der Diskussionsentwurf (`proposal.md`) und die Referenzimplementierung im Repository `github.com/naphets86/sharedlib` (Python-Paket `sl`), die gelesen, aber nicht ausgeführt wurden. Die Arbeit gibt ein formales Modell von Container, Suchordnung, Lader und Plattformadapter an und beweist die tragenden Aussagen: die Byte-Rundreise des Containers und seine Nichtkanonizität, die Struktur und Umgebungsunabhängigkeit der Suchordnung, die Charakterisierung erfolgreichen Ladens durch Zyklenfreiheit, die Initialisierungs- und Freigabereihenfolge, die Invariante der Referenzzählung, die Ausnahmesicherheit des Rollbacks, die Möglichkeit eines Deadlocks durch die globale Sperre sowie die Konvexität der Versionsbereiche. Zwei Sätze begründen Entwurfsentscheidungen des Vorschlags: Kein einzelner Maschinencode-Block kann zwei Aufrufkonventionen bedienen (Nichtziel 1), und die Auswahl von Dateiversionen mit Rückverfolgung ist NP-vollständig, während der Lader mit eindeutiger Auflösung in Zeit $O(V + E)$ arbeitet. Eine Konformitätsmatrix vergleicht jede Forderung des Entwurfs mit dem Quelltext. Vierzehn Befunde mit Verbesserungsvorschlägen, eine Zusammenfassung und ein sehr ausführlicher Ausblick schließen die Arbeit ab.

Inhaltsverzeichnis

I Ausgangslage **1**

1 Einführung **2**
1.1 Was aus der Vorarbeit übernommen wird . . . 2
1.2 Fragestellung . . . 3
1.3 Arten von Aussagen in dieser Arbeit . . . 3
1.4 Aufbau der Arbeit . . . 4

2 Plattformunterschiede und Problemlage **5**
2.1 Was sich heute unterscheidet . . . 5
2.2 Belege aus den zwei Projekten . . . 5
2.3 Ziele und Nichtziele . . . 6
2.4 Einordnung in die Wiederverwendungsformen . . . 7

3 Formale Grundlagen **8**
3.1 Plattformen, Targets, Versionen . . . 8
3.2 Metadaten und Container . . . 8
3.3 Umgebung, Dateisystem und Orakel . . . 9
3.4 Notation für Aussagen über Abläufe . . . 10

II Das Verfahren **11**

4 Anforderungen und Konformitätsmatrix **12**
4.1 Was „betriebssystemunabhängig“ bedeutet . . . 12
4.2 Konformitätsmatrix . . . 12

5 Das Container-Format **15**
5.1 Aufbau der Datei . . . 15
5.2 Rundreise . . . 16
5.3 Akzeptanzbedingungen . . . 16
5.4 Integrität und Kanonizität . . . 17

6 Suchordnung **19**
6.1 Namensauflösung . . . 19
6.2 Die Kandidatenliste . . . 19
6.3 Auswahl des Treffers . . . 21
6.4 Der gehärtete Modus . . . 21
6.5 Zerlegung von `SL_PATH` . . . 22

7 Der Lader-Kern **24**
7.1 Kosten und Charakterisierung des Erfolgs . . . 25
7.2 Operationsvertrag, Referenzzählung und Reihenfolgen . . . 26
7.3 Instanzen, Export und Nutzung . . . 29
7.4 Nebenläufigkeit . . . 30

8 Plattformadapter und Schichtung **31**
8.1 Der Adapter als Orakel . . . 31
8.2 Der Importgraph und die Schichten . . . 32
8.3 Wie weit trägt die Schichtung? . . . 32

III Umfeld des Verfahrens **35**

9 Codegenerierung und Build **36**
9.1 Ein durchgängiges Beispiel . . . 36
9.2 Eine Quelle der Wahrheit . . . 37
9.3 Signaturen und Typabbildung . . . 38
9.4 Build-Orchestrierung . . . 39

10 Versionen: Konvexität und Komplexität **40**
10.1 Die Bereichssprache . . . 40
10.2 Auswahl unter mehreren Kandidaten ist NP-vollständig . . . 41

11 Sicherheit **43**
11.1 Angreifermodell . . . 43
11.2 Die klassischen Angriffswege . . . 43
11.3 Einmaliges Lesen . . . 44

12 Anwendung auf die Beispielprojekte **45**
12.1 Vorher und nachher . . . 45
12.2 Metadaten für `wsat_shim` . . . 46
12.3 Ablauf eines `open` mit Abhängigkeit . . . 46
12.4 Der Test als Beleg: Reihenfolge der Abbildungen . . . 47

IV Prüfung **49**

13 Berichtete Ergebnisse **50**
13.1 Umfang und Abdeckung . . . 50
13.2 Welche Sätze die Tests belegen . . . 51
13.3 Einordnung der README-Aussagen . . . 52

14 Befunde und Verbesserungsvorschläge **53**

V Schluss **56**

15 Zusammenfassung **57**

16 Ausblick **59**
16.1 Die Befunde beheben: eine Reihenfolge . . . 59
16.2 Aussagen als Eigenschaftstests und mechanisch geprüfte Beweise . . . 60
16.2.1 Eigenschaftstests . . . 60
16.2.2 Mechanisch geprüfte Beweise . . . 60
16.3 Ein Lader mit Sperren je Schlüssel . . . 61
16.3.1 Zustände und Regeln . . . 61
16.3.2 Auswirkungen auf die Sätze . . . 62
16.4 Kanonischer Container und Signaturen . . . 62
16.4.1 Kanonische Kodierung . . . 62
16.4.2 Signierter Digest . . . 62
16.4.3 Weitere Fragen . . . 63
16.5 Eine Grammatik für `SL_PATH` . . . 63
16.6 Saubere Trennung zwischen Lader und Plattform . . . 63
16.6.1 Ein Lader-Kern in C . . . 64
16.6.2 Ein eigener Abbilder (Stufe 2) . . . 64
16.7 Native Abhängigkeiten, Laufzeit und ABI-Entwicklung . . . 64
16.7.1 Das Feld `runtime` mit Bedeutung füllen . . . 64
16.7.2 Symbolauflösung zwischen Bibliotheken . . . 65
16.7.3 Regeln der ABI-Entwicklung . . . 65
16.8 Signaturen, Besitz und weitere Sprachen . . . 65
16.9 Build-System und Werkzeugkette . . . 65
16.10 Versionen: Diagnose statt Suche . . . 66
16.11 Konformitätssuite mit sprachneutralen Vektoren . . . 66
16.12 Migration der Beispielprojekte . . . 66
16.13 Kostenmodell und Messungen . . . 67
16.14 Verbindung zum Subgraph-Programm . . . 67
16.15 Standardisierung und Betrieb . . . 68
16.16 Weitere Entwürfe des Entwurfs . . . 68
16.17 Offene Forschungsfragen . . . 68

Teil I

Ausgangslage

Kapitel 1

Einführung

Wer Software für mehrere Betriebssysteme ausliefert, muss dieselbe Bibliothek dreimal bauen, dreimal testen und dreimal dokumentieren. Das ist keine Eigenschaft der Aufgabe, sondern der Werkzeuge: Der Aufruf „lade die Bibliothek `wsat_shim`, finde `wsat_solve` und rufe sie auf“ ist auf allen Systemen derselbe, das Verfahren dahinter nicht. Die Arbeit zur Wiederverwendung des Subgraph Algorithmus [1] hat dies an zwei Projekten gezeigt, die C++-Code über eine Shared Library aus Python nutzen: der Biologie-Datenbank `github.com/naphets86/gen-db` und dem Projekt `github.com/naphets86/subgraph-sat-solver-econ`. Beide enthalten Code, der nichts mit ihrer Fachaufgabe zu tun hat, sondern nur Plattformunterschiede überbrückt (Abschnitt 2.2).

Aus dieser Beobachtung entstand der Diskussionsentwurf eines einheitlichen Formats `.sl` mit standardisierter Ladeprozedur und, als Antwort darauf, das Projekt `github.com/naphets86/sharedlib` mit dem Python-Paket `sl`: Container, Lader-Kern, Plattformadapter, Codegenerierung, Build-Orchestrierung und Kommandozeile, 14 Module mit zusammen 1 170 Zeilen Quelltext und 1 150 Zeilen Tests.

1.1 Was aus der Vorarbeit übernommen wird

Das Kapitel zum Laden der C/C++-Bibliothek in `csubgraph.tex` beschreibt die zwei Verfahren von Gen-DB und `subgraph-sat-solver-econ` und skizziert das Konzept eines einheitlichen Ladeverhaltens. Diese Arbeit *übernimmt* davon

- die Dokumentvorlage (Klasse, Pakete, Farben, Satz-, Definitions- und Beweisumgebungen, Tabellenspaltentypen `Y` und `L{w}`, Listing-Stil),
- die Einordnung in die sieben Wiederverwendungsformen (Abschnitt 2.4) und die Kennzeichnung der Aussagearten (Abschnitt 1.3),
- die Fragestellung des Ladens: Welche Plattformunterschiede gibt es, was lässt sich vereinheitlichen, was nicht?

und *führt aus*, was dort nur skizziert war: ein formales Modell, Beweise, die Konformitätsprüfung gegen den Quelltext, die Analyse von Sicherheit und Nebenläufigkeit, die Komplexität der Versionsauflösung sowie einen umfangreichen Ausblick.

Bemerkung 1.1 (Umfang der Vorlage)**.** `csubgraph.tex` bindet die Dateien `p1.tex`, `p2.tex` und `p3.tex` ein, die nicht Teil des Uploads waren. Das dortige Kapitel

zum Laden konnte daher nicht Zeile für Zeile übernommen werden. Soweit es sich in der hochgeladenen Hauptdatei, in der Einleitung der Vorlage und im `proposal.md` wiederfindet, ist es hier wiederverwendet; zum Quelltext von Gen-DB und `subgraph-sat-solver-econ` liegen nur die Belege aus `proposal.md` Abschnitt 2.1 vor, und die Arbeit beschränkt sich darauf.

1.2 Fragestellung

Die Arbeit beantwortet acht Fragen.

1. Was unterscheidet das Laden von Bibliotheken unter Windows, Linux und macOS, und welche dieser Unterschiede sind reine Plattformarbeit ohne fachlichen Gehalt? (Kapitel 2)

2. Was bedeutet „betriebssystemunabhängig“ präzise, und welche Forderungen des Entwurfs lassen sich als Aussagen über Programme formulieren? (Kapitel 3 und 4)

3. Wie ist das `.sl`-Containerformat aufgebaut, und welche Eigenschaften (Rundreise, Integrität, Kanonizität) hat es? (Kapitel 5)

4. Welche Suchordnung implementiert `sl`, wovon hängt sie ab, und welche Zusagen des Entwurfs hält sie? (Kapitel 6)

5. Wann ist das Laden erfolgreich, in welcher Reihenfolge laufen Initialisierung und Freigabe, und welche Garantien gelten für Referenzzählung, Rollback und Nebenläufigkeit? (Kapitel 7)

6. Wie weit trägt die Schichtung „nur der Adapter ist plattformspezifisch“? (Kapitel 8)

7. Wie hängen Codegenerierung, Build und Metadaten zusammen, und welche Zusagen folgen daraus? (Kapitel 9)

8. Warum löst der Lader keine Versionen auf, und was kostete es, wenn er es täte? (Kapitel 10)

1.3 Arten von Aussagen in dieser Arbeit

Wie in der Vorarbeit werden vier Arten von Aussagen unterschieden und im Text gekennzeichnet. Diese Arbeit führt den Quelltext *nicht* aus; die Art „Geprüft“ der Vorarbeit wird deshalb durch „Berichtet“ ersetzt.

- *Bewiesen:* Sätze, Lemmata und Korollare mit vollständigem Beweis. Sie gelten für das in Kapitel 3 definierte Modell; wo ein Beweis auf einer Annahme über die Umgebung beruht (etwa über `ctypes`), steht die Annahme ausdrücklich im Satz.

- *Aus dem Quelltext gelesen:* Eigenschaften, die sich aus dem hochgeladenen Stand von `sharedlib` ergeben (Signaturen, Standardwerte, Reihenfolgen). Sie sind durch Zeilenverweise belegt.

– *Berichtet:* Angaben der README und des Abdeckungsberichts unter `doc/coverage` (erzeugt am 21. Oktober 2026). Sie wurden nicht neu ermittelt.

– *Offen:* Aussagen, die einen Lauf, einen nicht vorliegenden Quelltext oder eine Entscheidung erfordern, etwa **[offen]**.

1.4 Aufbau der Arbeit

Die Arbeit hat fünf Teile (Abbildung 1.1).

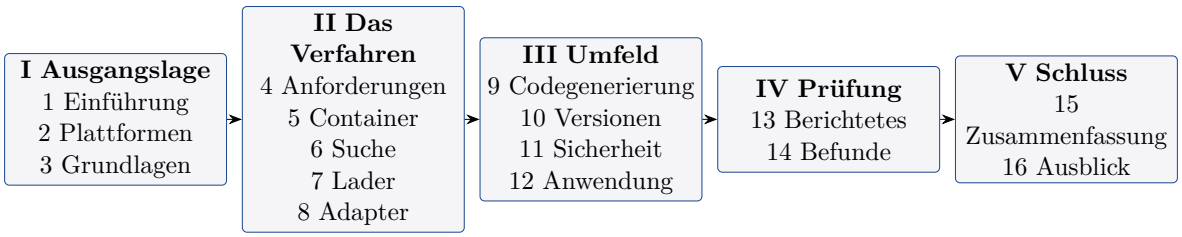


Abbildung 1.1: Gliederung der Arbeit. Teil II ist der formale Kern, Teil III stellt ihn in sein Umfeld, Teil IV vergleicht mit dem Quelltext, Teil V zieht Schlüsse.

Teil I legt die Problemlage und das formale Modell fest. Teil II untersucht die vier Bausteine des Verfahrens: Container, Suchordnung, Lader-Kern und Plattformadapter, jeweils mit Sätzen und Beweisen. Teil III behandelt Codegenerierung, die Komplexität der Versionsauflösung, die Sicherheit und die Anwendung auf die zwei Beispielprojekte. Teil IV stellt die Forderungen des Entwurfs dem Quelltext gegenüber und berichtet vierzehn Befunde. Teil V fasst zusammen und gibt einen umfangreichen Ausblick.

Kapitel 2

Plattformunterschiede und Problemlage

2.1 Was sich heute unterscheidet

Tabelle 2.1 stellt die Unterschiede zusammen, die `proposal.md` aufführt.

Aspekt	Windows	Linux	macOS
Dateiendung	`.dll`	`.so` (oft `.so.1.2.3`)	`.dylib` (Plug-ins `.so`)
Präfix	keines (MSVC) oder `lib` (MinGW)	`lib`	`lib`
Binärformat	PE/COFF	ELF	Mach-O
Lade-API	`LoadLibrary`	`dlopen`	`dlopen`
Standard-Sichtbarkeit	nichts exportiert ohne `dllexport`	alles sichtbar	alles sichtbar
Abhängigkeitssuche	DLL-Suchpfad, `PATH`, `add_dll_directory`	`RPATH`, `RUNPATH`, `LD_LIBRARY_PATH`	`@rpath`, `@loader_path`, `DYLD_*`
Initialisierung	`DllMain`	Konstruktoren, `.init_array`	Konstruktoren
Fehlermeldung	`GetLastError` (Zahl)	`dlerror` (Text)	`dlerror` (Text)
Entladen	`FreeLibrary`	`dlclose`	`dlclose`

Tabelle 2.1: Plattformunterschiede beim Laden dynamischer Bibliotheken (Auswahl nach `proposal.md`, Abschnitt 2).

2.2 Belege aus den zwei Projekten

Der Entwurf nennt sechs Stellen, an denen die Projekte Plattformcode enthalten; er enthält nach eigener Aussage keine Fachlogik. Die Angaben stammen aus `proposal.md` Abschnitt 2.1; der Quelltext der Projekte lag nicht vor.

1. `src/native.py` in `subgraph-sat-solver-econ`: `library_filename()` kennt vier Dateinamen (`wsat_shim.dll`, `libwsat_shim.dll`, `libwsat_shim.so`, `libwsat_shim.dylib`).

2. Dieselbe Datei ruft unter Windows `os.add_dll_directory` für jeden `PATH`-Eintrag auf.

3. `build_shim()` sucht zusätzlich `libwsat_shim.dll`, falls MinGW statt MSVC benutzt wurde.

4. `csubgraph_shim.cpp` in `gen-db` definiert das Makro `CSUB_EXPORT`.

5. `csubgraph_native.py` in `gen-db` linkt die MinGW-Laufzeit unter Windows statisch.

6. Die README von `subgraph-sat-solver-econ` beschreibt den Bau in PowerShell-Syntax und nennt vier mögliche Dateinamen.

Die Frage, wie das Laden in beiden Projekten im Einzelnen abläuft, wird nur so weit beantwortet, wie diese Belege reichen. Kapitel 12 zeigt, welche der sechs Stellen die Referenzimplementierung ersetzt.

2.3 Ziele und Nichtziele

Der Entwurf formuliert sieben Ziele: ein Namensschema `<name>.sl`; *eine* standardisierte, betriebssystemunabhängige Ladeprozedur, die durch eine Spezifikation und nicht durch das Verhalten des Systemladers definiert ist; ein ausdrückliches, überall gleiches Exportmodell; ein Metadatenblock; eine Suchpfadregel mit festgelegter Rangfolge; ein Fehlermodell mit maschinenlesbaren Codes, das auch fehlende Abhängigkeiten benennt; Koexistenz mit bestehenden Bibliotheken. Nichtziele sind: gleicher Maschinencode für alle Systeme und Prozessoren, die kurzfristige Ablösung von PE, ELF und Mach-O, und die Standardisierung von C++-ABIs (stabile Grenze ist die C-ABI).

Das erste Nichtziel ist keine Bescheidenheit, sondern eine Notwendigkeit. Der folgende Satz zeigt es in einem einfachen Maschinenmodell.

Definition 2.1 (Registermodell eines Aufrufs)**.** Ein *Code-Block* ohne Zugriff auf anderen Zustand als die Register ist eine Abbildung $B : \mathbb{Z}_{2^{64}}^{16} \to \mathbb{Z}_{2^{64}}$, die einem Registerzustand $r = (r_{\mathrm{rdi}}, r_{\mathrm{rcx}}, \dots)$ den Inhalt des Ergebnisregisters zuordnet. Die Funktion $\mathrm{id}(x) = x$ für ein 64-Bit-Argument wird von B unter einer *Aufrufkonvention* κ korrekt implementiert, wenn für jeden Registerzustand, in dem das erste Argument nach κ den Wert x hat, $B(r) = x$ gilt. Unter der Windows-x64-Konvention steht das erste Argument in r_{rcx}, unter der System-V-x86-64-Konvention in r_{rdi}.

Satz 2.1 (Kein universeller Code-Block)**.** Es gibt keinen Code-Block B im Sinne von Definition 2.1, der id sowohl unter der Windows-x64- als auch unter der System-V-x86-64-Konvention korrekt implementiert.

Proof. Angenommen, B implementiert id unter beiden Konventionen. Sei r der Registerzustand mit $r_{\mathrm{rdi}} = 1$, $r_{\mathrm{rcx}} = 2$ und beliebigen übrigen Registern. Unter Windows x64 hat das erste Argument in r den Wert $r_{\mathrm{rcx}} = 2$; also muss $B(r) = 2$ gelten. Unter System V hat es in r den Wert $r_{\mathrm{rdi}} = 1$; also muss $B(r) = 1$ gelten. Da B eine Funktion ist, folgt $2 = B(r) = 1$, ein Widerspruch. □

Der Satz gilt für Code, der allein aus den Registern liest. Ein Block, der zur Laufzeit am Betriebssystem erkennt, welche Konvention gilt, ist ausgenommen; er muss dazu zusätzlichen Zustand lesen und wäre damit gerade die Art von Plattformlogik, die die Abstraktion verbergen soll. Hinzu kommt der Datenmodellunterschied: `long` hat unter Windows (LLP64) 32, unter Linux (LP64) 64 Bit, so dass schon die Quelltext-Bedeutung einer Signatur nicht übertragbar ist. Die Konsequenz zieht der Entwurf selbst: vereinheitlicht werden *Schnittstelle, Container und Ladeprozedur*, nicht der Maschinencode. Der Container nimmt darum je Plattform und Architektur einen eigenen Payload auf (Kapitel 5).

2.4 Einordnung in die Wiederverwendungsformen

Die Vorarbeit unterscheidet sieben Formen der Wiederverwendung (Portierung, Verallgemeinerung, Neuimplementierung als Fallback, Bibliotheksnutzung, Prozessnutzung, Infrastruktur, Begriffliche). Das Paket `sl` fällt unter die Form *Infrastruktur*: Wiederverwendet wird nicht der Subgraph Algorithmus, sondern die Hülle, in der ein Algorithmus als Bibliothek ausgeliefert und geladen wird. Die Form hat damit die in der Vorarbeit beschriebene Eigenschaft: Änderungen am Algorithmus berühren sie nicht. Umgekehrt ändert sie die Form der *Bibliotheksnutzung*: Gen-DB und `subgraph-sat-solver-econ` würden die Bibliothek nicht mehr mit `ctypes.CDLL` über plattformabhängige Dateinamen laden, sondern mit `sl.open`. Die Kopplung bleibt dieselbe (gleicher Prozess, C-ABI); das Auffinden der Datei wird austauschbar.

Kapitel 3

Formale Grundlagen

Dieses Kapitel legt das Modell fest, auf das sich alle Sätze beziehen. Es bildet den Quelltext von `sl` so ab, dass jede Definition einer benannten Funktion entspricht; die Zuordnung steht jeweils im Text.

Abbildung 3.1 zeigt die vier Schichten, auf die sich das Modell bezieht.

3.1 Plattformen, Targets, Versionen

Definition 3.1 (Plattformraum und Target)**.** Sei $\mathcal{P} = \{\text{linux}, \text{macos}, \text{windows}\}$ und $\mathcal{A} = \{\text{x86_64}, \text{aarch64}, \text{x86}, \text{arm}\}$. Ein *Target* ist ein Paar $t = (p, a) \in \mathcal{P} \times \mathcal{A}$. Die Targets sind total geordnet durch die lexikographische Ordnung auf den Zeichenketten (Klasse `Target`, `order=True`).

Definition 3.2 (Version und Bereich)**.** Eine *Version* ist ein Tripel $v \in \mathbb{N}^3$ (Zeichenkette `major.minor.patch` aus Dezimalziffern), geordnet lexikographisch: $v < w$ genau dann, wenn die erste Komponente, in der sie sich unterscheiden, bei v kleiner ist. Eine *Einschränkung* ist ein Paar $(\bowtie, b)$ mit $\bowtie \in \{\geq, \leq, =, >, <\}$ und $b \in \mathbb{N}^3$; ein *Bereich* `s` ist eine endliche, durch Kommata getrennte Liste von Einschränkungen; die leere Liste (Zeichenketten `""` und `"*"`) ist erlaubt. v *erfüllt* `s` (`satisfies`), geschrieben $v \models \texttt{s}$, wenn $v \bowtie b$ für jede Einschränkung gilt. $S(\texttt{s}) = \{v : v \models \texttt{s}\}$.

3.2 Metadaten und Container

Definition 3.3 (Metadaten)**.** Ein *Metadatensatz* ist ein Tupel $M = (\nu, v, \alpha, T, D, X, \Sigma, \rho, \iota, \varphi)$ aus Name ν (ein Bezeichner `[A-Za-z_][A-Za-z0-9_]*`), Version v, ABI-Version $\alpha \in \mathbb{N}_{\geq 1}$, einer nichtleeren, duplikatfreien Targetmenge T, Abhängigkeiten D : Namen $\rightharpoonup$ Bereiche, einer duplikatfreien Liste exportierter Bezeichner X, Signaturen $\Sigma : X \rightharpoonup$ Text (nur für exportierte Symbole), Laufzeit $\rho \in \{\text{embedded}, \text{external}\}$ und optionalen Hook-Namen ι, φ (Init und Fini). Diese Bedingungen prüft `Metadata.__post_init__`.

Definition 3.4 (Container)**.** Ein *Container* ist ein Paar (M, P) aus einem Metadatensatz M und einer Abbildung $P : T \to \{0,1\}^*$ von der Targetmenge von M auf Byte-Folgen (die Payloads). Die Dateikodierung $\text{pack}(M, P)$ und ihre Umkehrung unpack sind in Kapitel 5 beschrieben.

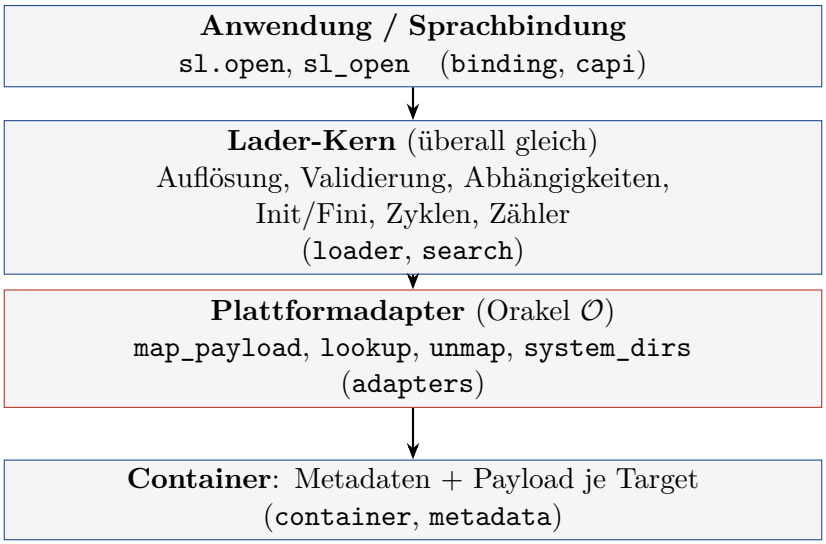


Abbildung 3.1: Schichten nach `proposal.md` Abschnitt 5.2 und README. Nach dem Entwurf liegt *alles* Betriebssystemspezifische im roten Adapter; Kapitel 8 prüft das.

3.3 Umgebung, Dateisystem und Orakel

Der Lader hängt von seiner Umgebung ab. Das Modell macht die Abhängigkeiten sichtbar.

Definition 3.5 (Umgebung des Laders)**.** Eine *Konfiguration* ist $\mathcal{K} = (E, R, A, h, \mathcal{F}, \mathcal{O})$ mit

- einer Umgebungsabbildung E von Variablennamen auf Zeichenketten (`env`),
- dem optionalen Verzeichnis des Aufrufers R (`requester_dir`) und dem optionalen Anwendungsverzeichnis A (`app_dir`),
- dem Flag h (`hardened`),
- einem Dateisystem $\mathcal{F}$: einer endlichen Menge regulärer Dateien mit Pfad, Inhalt und Rechten, und der Funktion realpath,
- einem *Adapter-Orakel* $\mathcal{O} = (\Sigma_{\text{sys}}, \text{map}, \text{lookup}, \text{unmap})$, das die vier plattformabhängigen Operationen des Adapters liefert: die Systemverzeichnisse $\Sigma_{\text{sys}}(E)$, das Abbilden eines Payloads, das Auffinden eines Symbols und das Freigeben (Klasse `PlatformAdapter`).

Annahme 3.1 (Determinismus der Umgebung)**.** Während einer Operation ändern sich E, $\mathcal{F}$ und die Antworten von $\mathcal{O}$ nicht, und $\mathcal{O}$ antwortet auf gleiche Anfragen gleich.

Die Annahme ist für die Beweise nötig und für die Wirklichkeit nur näherungsweise richtig; ihre Verletzung (Wettlauf zwischen Prüfen und Verwenden, Abschnitt 11.3) ist ein eigener Befund.

Definition 3.6 (Lader-Zustand und Handle)**.** Ein *Zustand* ist $s = (\Lambda, \sigma)$. Λ ist eine endliche partielle Abbildung von Schlüsseln (Pfaden nach `realpath`) auf *Handles*; ein Handle g trägt Metadaten M_g, native Ressource, Liste $\text{deps}(g)$ der Handles seiner Abhängigkeiten in Deklarationsreihenfolge und einen Zähler $\text{rc}(g) \in \mathbb{N}$. σ ist ein Keller von Schlüsseln (Feld `_loading`), der Pfad der gerade geladenen Bibliotheken.

Definition 3.7 (Abhängigkeitsgraph)**.** Für die Dateien $\mathcal{F}_{\text{sl}} \subseteq \mathcal{F}$, die gültige Container sind, sei $\text{res}(n, d)$ das Ergebnis von `locate` für den Namen n mit Aufrufer-Verzeichnis d (Kapitel 6), als Schlüssel; $\perp$, falls keine Datei gefunden wird. Der *Abhängigkeitsgraph* $G = (V, \to)$ hat $V = \mathcal{F}_{\text{sl}}$ und für jeden Eintrag $(n, \mathsf{s}) \in D_f$ eine Kante $f \to g$ mit $g = \text{res}(n, \text{dirname}(f))$, sofern $g \neq \perp$ ist; die Kante trägt die Beschriftung (n, s). Mehrfachkanten werden mitgezählt. G_r bezeichnet den von r aus erreichbaren Teilgraphen, V_r und E_r seine Knoten- und Kantenmengen.

3.4 Notation für Aussagen über Abläufe

Ein *Ablauf* ist die Folge der Aufrufe des Orakels und der Hooks, die eine Operation auslöst. Mit $\text{init}(g)$ und $\text{fini}(g)$ bezeichnen wir die Ereignisse „Init-Hook von g ist aufgerufen“ und „Fini-Hook ist aufgerufen“, mit $\text{map}(g)$ und $\text{unmap}(g)$ die Aufrufe des Orakels. Für Ereignisse e, e' bedeutet $e \prec e'$, dass e im Ablauf vor e' steht.

Teil II

Das Verfahren

Kapitel 4

Anforderungen und Konformitätsmatrix

4.1 Was „betriebssystemunabhängig“ bedeutet

Der Entwurf verlangt, dass die Ladeprozedur *betriebssystemunabhängig durch Abstraktion und Standardisierung* wird. Als Aussage über Programme heißt das: Zwei Plattformen dürfen sich nur dort unterscheiden, wo es ihr Adapter-Orakel tut.

Definition 4.1 (Beobachtung und Adapter-Parametrizität)**.** Die *Beobachtung* einer Operationsfolge ist die Folge der Rückgaben (Handle-Identitäten bis auf Umbenennung, Statuscodes, Felder `failed_name`, `searched`, `native_error`) zusammen mit dem Ablauf der Hook-Aufrufe. Ein Lader heißt *adapter-parametrisch*, wenn die Beobachtung bei gleicher Operationsfolge nur von $(E, R, A, h, \mathcal{F})$ und den Antworten des Orakels $\mathcal{O}$ abhängt, nicht aber vom Betriebssystem, auf dem er läuft.

Satz 8.2 zeigt, in welchem Sinn und mit welchen Ausnahmen die Referenzimplementierung diese Eigenschaft hat.

4.2 Konformitätsmatrix

Tabelle 4.1 stellt jede Forderung des Entwurfs (Abschnittsnummern nach `proposal.md`) dem Quelltext gegenüber. „Erfüllt“ heißt: aus dem Quelltext gelesen und, wo möglich, in einem Satz bewiesen. Die Befunde sind in Kapitel 14 ausgeführt.

Tabelle 4.1: Forderungen des Entwurfs und ihre Umsetzung in `sl`.

Forderung (Abschnitt)	Umsetzung	Status	Bezug
Namensauflösung, `.sl` anhängen, keine Plattformaffixe (5.3/1)	`search.resolve_name`	teilweise	Lemma 6.1; `lib`-Präfix als Bezeichner zulässig (B14)
Ortung, alle geprüften Kandidaten aufbewahren (5.3/2)	`search.locate`, Liste `tried`	teilweise	nur im Fehlerfall zurückgegeben (B14)

Forderung (Abschnitt)	Umsetzung	Status	Bezug
Containervalidierung vor dem Abbilden (5.3/3)	`loader._load`	erfüllt	Satz 7.2
Abhängigkeiten rekursiv, Zyklen erkennen (5.3/4)	`loader._open_dependency`, `_loading`	erfüllt	Satz 7.2
Abbilden über Plattformadapter (5.3/5)	`adapters.map_payload` über temporäre Datei	erfüllt (Stufe 1)	Kapitel 8
Init: Abhängigkeiten zuerst, einmal; `DllMain`/Konstruktoren umhüllt (5.3/6)	`loader._run_init`; Konstruktoren laufen im Systemlader	teilweise	Satz 7.4; B14
Exporttabelle, nur deklarierte Symbole (5.3/7)	`Loader.symbol`	erfüllt (API-Ebene)	Satz 7.7
Referenzgezählte Handles, gleiche Instanz (5.3/8)	`Handle.refcount`, `_loaded`	erfüllt	Satz 7.3; B13
Freigabe in umgekehrter Abhängigkeitsordnung (5.3/9)	`loader._release`	erfüllt	Satz 7.4
Suchordnung: explizit, `SL_PATH`, Aufrufer, Anwendung, System (5.4)	`search.candidates`	erfüllt; App-Pfad nur falls gesetzt	Satz 6.1
Ein einheitliches Trennzeichen mit Maskierung (5.4)	`search.split_sl_path`	abweichend	Satz 6.5; B1
Arbeitsverzeichnis, `PATH`, `LD_LIBRARY_PATH` nie (5.4)	`search.candidates`	erfüllt unter Bedingung	Satz 6.2; B2
API `sl_open`, `sl_symbol`, `sl_info`, `sl_close` (5.5)	`capi`, `binding`	erfüllt (Python); C-ABI des Laders fehlt	Kapitel 8
Fehlermodell: Codes, `failed_name`, `searched`, `native_error` (5.6)	`errors.SLError`, `capi.ErrorInfo`	erfüllt; mehrzeiliger Text fehlt	B14
`open`/`close` threadsicher (5.7)	`RLock`	erfüllt	Satz 7.8
Init nie unter einer Sperre, die Nutzercode betreten kann (5.7)	`RLock` wird gehalten	nicht erfüllt	Satz 7.9; B4
Zyklus bei `open` im Init als `CYCLE` (5.7)	`_loading`	erfüllt	Satz 7.2

Forderung (Abschnitt)	Umsetzung	Status	Bezug
Export standardmäßig verborgen, ein Makro (5.8)	`SL_EXPORT`, `-fvisibility=hidden`	erfüllt	Kapitel 9
Konformitätssuite (5.9)	Unit-Tests mit `FakeAdapter`	teilweise	Kapitel 13
Metadatenfelder (6.2)	`metadata.Metadata`	erfüllt; `runtime` ohne Wirkung	B11
Echter Container, Fat-Binary (6.3)	`container`	erfüllt	Kapitel 5
Legacy-Modus für `.dll`/`.so`/`.dylib` (8.3)	–	fehlt	B14
Hash und Signatur (10)	SHA-256 je Payload, keine Signatur	teilweise	Satz 5.2; B7
Gehärteter Modus, fail closed (10)	`hardened`	teilweise	Satz 6.4; B3
Bei widersprüchlichen Kandidaten Fehler (10)	erster Treffer gewinnt	abweichend	Satz 6.3

Kapitel 5

Das Container-Format

5.1 Aufbau der Datei

Ein `.sl`-Container besteht aus Kopf, Metadaten, Payload-Tabelle und Payload-Bereich (Abbildung 5.1). Die Kodierung steht in `container.py`.

Listing 5.1: Layout und Kodierung (`container.py`, Z. 12–15, 54–65).

```
MAGIC = b"\x89SLIB\r\n\x1a"  # genau 8 Byte
CONTAINER_VERSION = 1
_HEADER = struct.Struct("<8sHII")       # magic, version, meta_len, entry_count
_ENTRY = struct.Struct("<16s16sQQ32s")  # platform, arch, offset, length, sha256

def pack(metadata: Metadata, payloads: dict) -> bytes:
    if set(payloads) != set(metadata.targets):
        raise _bad("payload targets differ from metadata targets")
    meta_bytes = json.dumps(metadata.to_dict(), sort_keys=True).encode("utf-8")
    table, blob = [], bytearray()
    for target in sorted(payloads):
        data = bytes(payloads[target])
        table.append(_ENTRY.pack(target.platform.encode("ascii"), target.arch.encode("ascii"),
                                 len(blob), len(data), hashlib.sha256(data).digest()))
        blob += data
    head = _HEADER.pack(MAGIC, CONTAINER_VERSION, len(meta_bytes), len(table))
    return head + meta_bytes + b"".join(table) + bytes(blob)
```

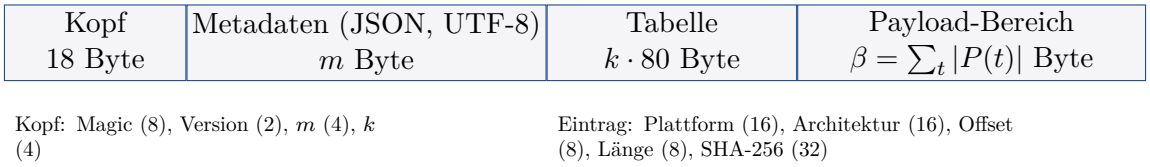


Abbildung 5.1: Byte-Layout eines Containers. Ein Eintrag der Tabelle hat $16+16+8+8+32 = 80$ Byte, der Kopf $8+2+4+4 = 18$ Byte (Little Endian, ohne Auffüllbytes wegen des Präfixes `<`).

Lemma 5.1 (Größenformel)**.** Für gültige (M, P) mit $k = |T|$ Targets und Metadaten-Kodierung der Länge m gilt

$$|\mathrm{pack}(M, P)| = 18 + m + 80\,k + \sum_{t \in T} |P(t)|.$$

Proof. Der Kopf hat nach der Formatzeichenkette `<8sHII` $8 + 2 + 4 + 4 = 18$ Byte, ein Tabelleneintrag nach `<16s16sQQ32s` $16 + 16 + 8 + 8 + 32 = 80$ Byte. `pack` legt genau k Einträge an, eine Zeile je Target aus `sorted(payloads)`, und hängt die Payloads lückenlos an den Payload-Bereich. □

5.2 Rundreise

Lemma 5.2 (Metadaten-Rundreise)**.** Für jeden gültigen Metadatensatz M gilt `Metadata.from_dict(json.loads(json.dumps(M.to_dict(), sort_keys=True))) = M`.

Proof. `to_dict` erzeugt ein Objekt aus Zeichenketten, einer ganzen Zahl, Listen, Wörterbüchern und `None`. Solche Werte werden von JSON ohne Verlust abgebildet; `None` wird zu `null` und zurück, die Schlüsselreihenfolge ändert die Werte nicht. Die Felder werden in `from_dict` einzeln zurückgewonnen: Name, Version, ABI-Version, Laufzeit, Hooks unverändert; die Targets als Tupel in derselben Reihenfolge, in der `to_dict` sie ausgibt; `exports` als Tupel derselben Liste; `dependencies` und `signatures` als Wörterbücher mit gleicher Abbildung (die Gleichheit von Wörterbüchern ignoriert die Einfügereihenfolge). Der Konstruktor prüft dieselben Bedingungen wie beim ersten Aufbau und akzeptiert sie wieder, weil sie nur von diesen Werten abhängen. Da die Dataclass `Metadata` komponentenweise vergleicht, folgt die Gleichheit. □

Satz 5.1 (Rundreise des Containers)**.** Sei (M, P) ein Container im Sinne von Definition 3.4. Dann liefert `unpack(pack(`M, P`))` einen Container mit Metadaten M, und `payload_for`(p, a) gibt für jedes Target $(p, a) \in T$ genau $P(p, a)$ zurück.

Proof. Sei $d = \mathrm{pack}(M, P)$, m die Länge der Metadaten-Kodierung und $k = |T|$. Der Kopf wird mit Magic, Version 1, m und k gepackt; `unpack` liest sie zurück, die Prüfungen auf Länge, Magic und Version bestehen. Nach Lemma 5.1 ist $18 + m + 80k \le |d|$, die Tabelle liegt also vollständig in d. Die Metadaten werden nach Lemma 5.2 zu M zurückgelesen. Seien $t_0 < \cdots < t_{k-1}$ die Targets in aufsteigender Ordnung. Eintrag i trägt den Offset $o_i = \sum_{j<i} |P(t_j)|$ und die Länge $|P(t_i)|$; der Payload-Bereich `blob` ist $d[18 + m + 80k :]$ und hat die Länge $\beta = \sum_j |P(t_j)|$, so dass $o_i + |P(t_i)| \le \beta$ gilt. Die Namen der Targets sind ASCII und kürzer als 16 Byte (längste Namen `windows` und `aarch64` mit 7 Byte), das Auffüllen mit Nullbytes und das Abschneiden mit `rstrip` sind also umkehrbar. Die Menge der gelesenen Targets ist T, die Prüfung auf Gleichheit der Mengen besteht. `payload_for` schneidet $\mathrm{blob}[o_i : o_i + |P(t_i)|] = P(t_i)$ aus und vergleicht dessen SHA-256 mit dem gespeicherten Wert, der genau dieser Hash ist. □

5.3 Akzeptanzbedingungen

Lemma 5.3 (Prüfungen von `unpack`)**.** Wenn `unpack`(d) einen Container zurückgibt, dann gilt:

(S1) $|d| \geq 18$;

(S2) die ersten 8 Byte sind das Magic;

(S3) die Version im Kopf ist 1;

(S4) $18 + m + 80k \leq |d|$ für die Werte m, k des Kopfes;

(S5) die Bytes $d[18 : 18 + m]$ sind ein JSON-Dokument, aus dem `Metadata.from_dict` einen gültigen Metadatensatz bildet;

(S6) jeder der k Tabelleneinträge nennt ASCII-Namen eines gültigen Targets, und es gilt $o + \ell \leq \beta$ für Offset o und Länge ℓ;

(S7) die Menge der Targets der Tabelle ist gleich der Targetmenge der Metadaten.

Proof. Jede Bedingung entspricht einer `raise`-Anweisung in `unpack` (Zeilen 69–97): zu kurze Datei, falsches Magic, falsche Version, abgeschnittener Container, unlesbare Metadaten, Nicht-ASCII-Namen und ungültiges `Target` (die Konstruktion löst `INVALID_CONTAINER` aus), Payload außerhalb, abweichende Targetmengen. Ein Rückgabewert entsteht nur, wenn keine dieser Anweisungen ausgeführt wurde. □

5.4 Integrität und Kanonizität

Satz 5.2 (Integrität der Payloads)**.** Sei $f = \mathrm{pack}(M, P)$, $f' \neq f$ gleich lang, und f' unterscheide sich von f nur in Bytes des Payload-Bereichs, die zum Target t gehören. Wenn `unpack`(f') gelingt, dann wirft `payload_for`(t) auf f' eine Ausnahme `INVALID_CONTAINER`, außer es wurde eine Kollision von SHA-256 gefunden, also zwei verschiedene Byte-Folgen mit gleichem Hash.

Proof. Die Tabelle ist unverändert, der gespeicherte Hash also der von $P(t)$. Die in f' ausgeschnittene Folge $P'(t)$ hat dieselbe Länge und ist von $P(t)$ verschieden. `payload_for` vergleicht $\mathrm{SHA256}(P'(t))$ mit dem gespeicherten Wert $\mathrm{SHA256}(P(t))$ und wirft bei Ungleichheit. Gleichheit bei $P'(t) \neq P(t)$ wäre eine Kollision. □

Bemerkung 5.1 (Was der Hash nicht schützt)**.** Der Hash liegt in derselben Datei wie der Payload und schützt deshalb nur gegen *zufällige* Veränderung des Payloads. Ein Angreifer, der den Payload ändert, ändert auch den Hash. Metadaten, Tabelle und Kopf sind von keinem Hash erfasst: Aus `"version": "1.2.0"` wird durch Ändern eines Bytes `"1.3.0"`, und `unpack` akzeptiert die Datei. Echtheit verlangt eine Signatur (Abschnitt 16.4).

Satz 5.3 (Nichtkanonizität)**.** `unpack` ist auf der Menge der akzeptierten Dateien nicht injektiv bis auf Beobachtung: Es gibt verschiedene Dateien $d \neq d'$, für die `unpack` Container mit gleichen Metadaten und gleichen `payload_for`-Ergebnissen liefert. Insbesondere ist $\mathrm{pack} \circ \mathrm{unpack} \neq \mathrm{id}$.

Proof. Sei $d = \mathrm{pack}(M, P)$. Folgende Dateien werden akzeptiert und sind beobachtbar gleich:

(a) $d' = d \,\|\, \texttt{00}$: der angehängte Byte liegt im Payload-Bereich hinter allen Einträgen; keine Bedingung aus Lemma 5.3 verbietet überzählige Bytes.

(b) Die JSON-Kodierung der Metadaten mit anderem Leerraum oder anderer Schlüsselreihenfolge: `json.loads` liefert dasselbe Objekt; die Längenangabe m im Kopf wird entsprechend angepasst.

(c) Die Tabelleneinträge in anderer Reihenfolge: `unpack` sammelt sie in einem Wörterbuch nach Target; die Reihenfolge geht nicht ein.

(d) Zwei Targets mit gleichem Payload verweisen auf denselben Bereich: (S6) fordert nur $o + \ell \leq \beta$, nicht Überlappungsfreiheit, und der Hash stimmt für beide.

(e) Ein doppelt eingetragenes Target: das Wörterbuch behält den letzten Eintrag, (S7) vergleicht Mengen.

In jedem Fall unterscheiden sich die Bytes von d, die Beobachtung aber nicht. Da pack nur die kanonische Form erzeugt, folgt $\mathrm{pack}(\mathrm{unpack}(d')) \neq d'$. □

Die Folge für eine spätere Signatur ist praktisch: Wer eine Signatur über die Datei bildet, muss entweder eine kanonische Form erzwingen oder über wohldefinierte Teile (Metadaten-Bytes, Tabelle, Payload-Hashes) signieren. Beides ist in der Referenzimplementierung nicht angelegt (Befund 14.7).

Kapitel 6

Suchordnung

6.1 Namensauflösung

Lemma 6.1 (Zulässige Namen)**.** Sei x eine Zeichenkette ohne `/` und `\`. `resolve_name`(x) liefert genau dann ein Ergebnis, wenn $x \in I \cup I \cdot$ `.sl` mit $I = \{$`[A-Za-z_][A-Za-z0-9_]*`$\}$; das Ergebnis ist `(base + ".sl", False)`. Enthält x ein `/` oder `\`, so ist der Name ein expliziter Pfad: `(x', True)` mit $x' = x$, falls x auf `.sl` endet, sonst $x' = x +$ `.sl`. In allen anderen Fällen wird `NOT_FOUND` geworfen.

Proof. Ohne Trennzeichen entfernt der Code genau ein endendes `.sl` und prüft die Basis mit der regulären Ausdrucksform I. Eine Basis in I enthält keinen Punkt, so dass `libwsat.so`, `wsat.dll` und `wsat.sl.sl` ausscheiden, `wsat` und `wsat.sl` bestehen. Die leere Zeichenkette liegt nicht in I. Der explizite Pfad folgt direkt aus dem ersten `if`. □

Bemerkung 6.1. Plattformsuffixe (`.so`, `.dll`, `.dylib`) sind damit *syntaktisch* ausgeschlossen, Plattformpräfixe aber nicht: `libwsat` ist ein Bezeichner und wird als logischer Name `libwsat.sl` gesucht. Die Forderung des Entwurfs, das Präfix sei nicht Teil des logischen Namens, ist damit eine Konvention, keine Prüfung (B14).

6.2 Die Kandidatenliste

Listing 6.1: Kandidatenliste (`search.py`, Zeilen 97–110).

```
def candidates(filename, explicit, *, env, requester_dir, app_dir, adapter,
    hardened) -> list:
    if explicit:
        return [filename]
    out = []
    if not hardened:
        out += [_join(d, filename) for d in split_sl_path(env.get("SL_PATH"
, ""))
                if _is_absolute(d)]
    if requester_dir:
        out += [_join(requester_dir, filename),
                _join(requester_dir, "sl", filename)]
    if app_dir:
        out.append(_join(app_dir, filename))
    out += [_join(d, filename) for d in adapter.system_dirs(env)]
    return list(dict.fromkeys(out))
```

Definition 6.1 (Kandidatenliste)**.** Für Dateinamen f, $e = \texttt{explicit}$ und eine Konfiguration $\mathcal{K}$ sei $\mathcal{C}(f, e, \mathcal{K})$ die von Listing 6.1 berechnete Liste. Mit $\mathrm{dd}(L)$ bezeichnen wir die Liste L ohne spätere Wiederholungen (`dict.fromkeys`).

Satz 6.1 (Struktur und Umgebungsabhängigkeit)**.** Für $e =$ falsch gilt $\mathcal{C}(f, e, \mathcal{K}) = \mathrm{dd}(L_{\mathrm{env}} \,\|\, L_{\mathrm{req}} \,\|\, L_{\mathrm{app}} \,\|\, L_{\mathrm{sys}})$ mit

$$L_{\mathrm{env}} = [\, d/f : d \in \mathrm{split}(E[\texttt{SL_PATH}]),\ d \text{ absolut} \,] \ \text{ falls } h = \text{falsch},\ \text{sonst leer},$$
$$L_{\mathrm{req}} = [\, R/f,\ R/\texttt{sl}/f \,] \ \text{ falls } R \text{ gesetzt}, \quad L_{\mathrm{app}} = [\, A/f \,] \ \text{ falls } A \text{ gesetzt},$$
$$L_{\mathrm{sys}} = [\, d/f : d \in \Sigma_{\mathrm{sys}}(E) \,].$$

Für $e =$ wahr ist die Liste $[f]$. Die Liste hängt von der Umgebung E nur über $E[\texttt{SL_PATH}]$ (bei $h =$ falsch) und über $\Sigma_{\mathrm{sys}}(E)$ ab. Bei den Adaptern für Linux und macOS ist Σ_{sys} konstant; beim Windows-Adapter hängt es von `ProgramData` und `ProgramFiles` ab. Von `PATH`, `LD_LIBRARY_PATH` und `DYLD_LIBRARY_PATH` hängt die Liste in keinem Fall ab.

Proof. Die Gleichung ist der Wortlaut von Listing 6.1. Auf E greift der Code nur an zwei Stellen zu: `env.get("SL_PATH", "")` im Zweig $\neg h$ und `adapter.system_dirs(env)`. Die Methoden `LinuxAdapter.system_dirs` und `DarwinAdapter.system_dirs` geben konstante Listen zurück (`adapters.py` Zeilen 75 und 82), `WindowsAdapter.system_dirs` liest `env[key]` für `ProgramData` und `ProgramFiles` (Zeile 89). Andere Schlüssel werden nicht gelesen. □

Satz 6.2 (Freiheit vom Arbeitsverzeichnis)**.** Sei $e =$ falsch, seien R und A nicht gesetzt oder absolut, und seien alle Elemente von $\Sigma_{\mathrm{sys}}(E)$ absolut. Dann ist jeder Kandidat in $\mathcal{C}(f, e, \mathcal{K})$ ein absoluter Pfad, und die Liste sowie das Ergebnis von `locate` hängen nicht vom aktuellen Arbeitsverzeichnis ab. Ohne die Voraussetzung über R gilt das nicht.

Proof. Die Elemente von L_{env} sind nach dem Filter `_is_absolute` absolut. `_join` hängt an eine absolute Basis nur Komponenten an, das Ergebnis beginnt mit der Basis und ist absolut. Für L_{req}, L_{app} und L_{sys} gilt das nach Voraussetzung. Ein absoluter Pfad wird vom Dateisystem unabhängig vom Arbeitsverzeichnis aufgelöst. Gegenbeispiel: Für $R =$ `rel` enthält die Liste `rel/x.sl`, das relativ zum Arbeitsverzeichnis aufgelöst wird. □

Der Lader erzwingt die Voraussetzung nicht: `Loader.__init__` übernimmt `requester_dir` und `app_dir` unverändert (Befund 14.2). Für Abhängigkeiten ist sie erfüllt, weil das Aufruferverzeichnis aus dem Schlüssel (`realpath`) gebildet wird.

Lemma 6.2 (Entdoppelung)**.** $\mathrm{dd}(L)$ enthält jedes Element von L genau einmal, in der Reihenfolge seines ersten Auftretens in L.

Proof. `dict.fromkeys(L)` fügt die Elemente von L der Reihe nach als Schlüssel ein; ein vorhandener Schlüssel wird nicht erneut eingefügt und behält seine Position. Die Sprachdefinition von Python garantiert seit Version 3.7 die Einfügereihenfolge von Wörterbüchern. □

6.3 Auswahl des Treffers

Listing 6.2: Auswahl (`search.py`, Zeilen 130–145).

```
def locate(name_or_path, *, env, requester_dir, app_dir, adapter, hardened=
    ↪ False) -> tuple:
    filename, explicit = resolve_name(name_or_path)
    tried = []
    for path in candidates(filename, explicit, env=env, requester_dir=
    ↪ requester_dir,
                           app_dir=app_dir, adapter=adapter, hardened=
    ↪ hardened):
        tried.append(path)
        if os.path.isfile(path):
            dir_to_check = os.path.dirname(os.path.abspath(path))
            if hardened and _world_writable(dir_to_check):
                raise SLError(SLStatus.ACCESS_DENIED,
                              f"'{path}' lies in a world-writable directory
    ↪ ",
                              failed_name=name_or_path, searched=tried)
            return os.path.abspath(path), tried
    raise SLError(SLStatus.NOT_FOUND, f"library '{name_or_path}' not found"
    ↪ ,
                  failed_name=name_or_path, searched=tried)
```

Satz 6.3 (Erster Treffer gewinnt)**.** Sei $\mathcal{C} = [p_1, \ldots, p_n]$ die Kandidatenliste und i der kleinste Index, für den p_i eine reguläre Datei ist. Dann gilt bei $h =$ falsch: `locate` gibt $(\mathrm{abspath}(p_i), [p_1, \ldots, p_i])$ zurück. Gibt es kein solches i, wird `NOT_FOUND` mit `searched`$= [p_1, \ldots, p_n]$ geworfen. Die Zahl der Dateisystemprüfungen ist höchstens $n \le |L_{\mathrm{env}}| + 3 + |L_{\mathrm{sys}}|$. Spätere Kandidaten werden nach einem Treffer nicht mehr betrachtet, auch wenn der Treffer in der weiteren Prüfung (Version, ABI) scheitert.

Proof. Die Schleife durchläuft die Liste in Reihenfolge, fügt jeden Pfad vor der Prüfung an `tried` an und kehrt beim ersten Treffer zurück. Erschöpft sich die Schleife, wird mit der vollständigen Liste geworfen. Die Schranke folgt aus Satz 6.1. Die Prüfung von Version und ABI findet erst im Lader nach der Rückgabe statt; ein Fehler dort ruft `locate` nicht erneut mit späteren Kandidaten auf. □

Die letzte Aussage hat Folgen: Der Entwurf verlangt (Abschnitt 10) bei widersprüchlichen Kandidaten einen Fehler statt einer stillen Wahl. Die Implementierung wählt still den ersten. Das ist für die Reproduzierbarkeit gut (Ergebnis folgt der festen Rangfolge), für die Erkennung von Verwechslungen schwach; Kapitel 10 zeigt, dass eine Suche über *alle* Kandidaten mit Rückverfolgung das Problem schwer machen würde.

6.4 Der gehärtete Modus

Satz 6.4 (Gehärteter Modus)**.** Sei $h =$ wahr und $e =$ falsch. Dann gilt:

(i) `SL_PATH` hat keinen Einfluss auf die Kandidatenliste.

(ii) Gibt `locate` einen Pfad zurück, so hat das Verzeichnis des Treffers auf einem POSIX-System das Bit `S_IWOTH` nicht gesetzt.

(iii) Liegt der erste existierende Kandidat in einem solchen Verzeichnis, wird `ACCESS_DENIED` geworfen, auch wenn ein späterer Kandidat in einem sicheren Verzeichnis liegt (*fail closed*).

Proof. (i) Nach Satz 6.1 ist L_{env} bei $h =$ wahr leer. (ii) Die Rückgabe erfolgt nur, wenn die Bedingung `hardened and _world_writable(dir)` falsch war; unter POSIX gibt `_world_writable` den Wert von `st_mode & S_IWOTH` zurück, bei einem Fehler von `os.stat` dagegen `False`. Im Erfolgsfall hat `stat` also entweder das Bit nicht gesetzt gefunden oder ist gescheitert. (iii) Die Prüfung erfolgt beim ersten Treffer; bei Zutreffen wird sofort geworfen, spätere Kandidaten werden nicht betrachtet. □

Die Grenzen dieses Satzes sind Gegenstand von Befund 14.3: Geprüft wird nur das unmittelbare Verzeichnis, nur das Bit für *others*, nicht Besitzer, Gruppenrechte oder übergeordnete Verzeichnisse; bei einem `stat`-Fehler wird die Datei zugelassen (fail open), und unter Windows enthält der Code eine auf einen Testnamen zugeschnittene Sonderregel (Zeilen 120–123).

6.5 Zerlegung von `SL_PATH`

Listing 6.3: `split_sl_path` (`search.py`, Zeilen 14–65, gekürzt).

```
def split_sl_path(raw: str) -> list[str]:
    if not raw:
        return []
    # Windows Semikolon-Trennliste unterstützen
    if ";" in raw:
        return [p for p in raw.split(";") if p]
    parts, current, i, n = [], [], 0, len(raw)
    while i < n:
        # Windows-Laufwerksbuchstabe (z. B. C:\)
        if (i == 1 and raw[0].isalpha() and raw[1] == ":"
                and (n == 2 or raw[2] in ("/", "\\"))):
            current.append(":"); i += 1; continue
        if raw[i] == "\\" and i + 1 < n:
            if raw[i + 1] == ":":  current.append(":");  i += 2; continue
            if raw[i + 1] == "\\": current.append("\\"); i += 2; continue
        if raw[i] == ":":
            token = "".join(current)
            if token: parts.append(token)
            current = []; i += 1
        else:
            current.append(raw[i]); i += 1
    if current:
        token = "".join(current)
        if token: parts.append(token)
    return parts
```

Der Entwurf verlangt ein *einziges* Trennzeichen mit festgelegter Maskierung, damit `SL_PATH` auf allen Systemen gleich gelesen wird (Abschnitt 5.4 und offene Frage in Abschnitt 14). Die Umsetzung hat zwei Betriebsarten, die der *Inhalt* der Zeichenkette wählt, nicht die Plattform.

Definition 6.2 (Maskierung und Verbindung)**.** Für eine Zeichenkette p sei $\mathrm{esc}(p)$ das Ergebnis, in dem jedes `\` durch `\\` und jedes `:` durch `\:` ersetzt wird. Für eine Liste $(p_1, \dots, p_k)$ sei $\mathrm{join}(p_1, \dots, p_k) = \mathrm{esc}(p_1) : \cdots : \mathrm{esc}(p_k)$.

Satz 6.5 (Zerlegung von `SL_PATH`)**.** Sei split die Funktion `split_sl_path`.

(a) split(*raw*) ist eine Funktion von *raw* allein; sie hängt weder von der Plattform noch von der Umgebung ab, und ihre Laufzeit ist $O(|raw|)$.

(b) Enthält *raw* ein Semikolon, so ist split(*raw*) die Liste der nichtleeren Teile von `raw.split(";")`. Jeder Teil ist semikolonfrei und wird nicht entmaskiert; der Doppelpunkt trennt dann nicht.

(c) Seien $p_1, \ldots, p_k$ nichtleere, semikolonfreie Zeichenketten. Gilt nicht zugleich $k \geq 2$, p_1 ist ein einzelner Buchstabe und das erste Zeichen von esc(p_2) ist `/` oder `\`, so ist split(join($p_1, \ldots, p_k$)) = $(p_1, \ldots, p_k)$.

(d) Kein Eintrag, der ein Semikolon enthält, ist darstellbar: Für kein *raw* enthält split(*raw*) einen Eintrag mit Semikolon. Insbesondere ist die Abbildung split *nicht* surjektiv auf Listen nichtleerer Zeichenketten.

(e) Dieselbe Zeichenkette kann je nach Inhalt Verschiedenes bedeuten: split(`a:b`) = (`a`, `b`), aber split(`a:b;c`) = (`a:b`, `c`).

Proof. (a) Der Code liest nur *raw* und besteht aus einer Schleife, die in jedem Durchlauf mindestens ein Zeichen konsumiert. (b) Der erste `if` gibt in diesem Fall sofort zurück. (d) folgt aus (b): Teile von `raw.split(";")` enthalten kein Semikolon, und ein Semikolon in einem Eintrag lässt *raw* ein Semikolon enthalten, so dass nach (b) zerlegt wird. (e) ist Anwendung von (b) und (c) mit den angegebenen Zeichenketten.

(c) Da kein p_i ein Semikolon enthält, enthält auch *raw* = join(...) keines; der Zweig aus (b) wird nicht betreten. Jede Einheit der Kodierung ist entweder ein Zeichen, das weder `\` noch `:` ist, ein Maskierungspaar `\\` oder `\:`, oder ein Trennzeichen `:` zwischen zwei Einträgen. In esc-Ausgaben steht jedes `\` am Anfang eines Paares, ein einzelnes `\` kommt nicht vor, und ein unmaskiertes `:` kommt nur als Trennzeichen vor. Wir zeigen durch Induktion über die Zahl der verarbeiteten Einheiten: Steht die Schleife an einer Einheitengrenze, so enthält `current` die entmaskierten Zeichen des laufenden Eintrags und `parts` die fertigen Einträge. Beginn: leer. Schritt: Ein gewöhnliches Zeichen wird angehängt; ein Paar `\:` bzw. `\\` wird durch den zweiten bzw. dritten `if`-Zweig als `:` bzw. `\` angehängt und überspringt zwei Zeichen; ein Trennzeichen schließt den Eintrag ab (er ist nichtleer) und beginnt einen neuen. Der Laufwerkszweig könnte nur bei $i = 1$ eingreifen, wenn `raw[0]` ein Buchstabe und `raw[1]` ein `:` ist. Dann ist `raw[0]` einzelnes Zeichen vor einem unmaskierten Doppelpunkt, also p_1 ein einzelner Buchstabe und $k \geq 2$; der Zweig greift zusätzlich nur, wenn $n = 2$ (unmöglich bei $k \geq 2$) oder `raw[2]`, das erste Zeichen von esc(p_2), `/` oder `\` ist. Das ist durch die Voraussetzung ausgeschlossen. Am Ende hängt der Code den letzten nichtleeren Eintrag an. Also ist das Ergebnis $(p_1, \ldots, p_k)$. □

Beispiel 6.1 (Zwei Laufwerke)**.** Für *raw* = `D:\a:C:\b` erkennt der Laufwerkszweig nur das erste Laufwerk. Die Zerlegung liefert (`D:\a`, `C`, `\b`) und nicht die beiden Pfade. Der zweite Laufwerksbuchstabe muss maskiert werden (`C\:\\b`) oder die Zeichenkette muss ein Semikolon als Trenner verwenden.

Der Satz zeigt, dass die README-Formulierung „Trennzeichen `:` unter POSIX oder `;` unter Windows“ die Implementierung ungenau beschreibt: Die Wahl trifft die Zeichenkette, nicht das System. Das ist gerade für die Forderung der Plattformunabhängigkeit günstig (Teil (a)), aber nicht dokumentiert, und der Zweig mit Semikolon ist im Abdeckungsbericht als nie ausgeführt ausgewiesen (Befund 14.1).

Kapitel 7

Der Lader-Kern

Der Lader-Kern setzt die neun Schritte des Entwurfs (Abschnitt 5.3) um. Listing 7.1 zeigt die drei Funktionen, auf denen alle folgenden Sätze beruhen.

Listing 7.1: `Loader._open`, `_load` und `_release` (`loader.py`, gekürzt).

```
def _open(self, name_or_path, spec, requester_dir) -> Handle:
    path, _tried = locate(name_or_path, env=self.env, requester_dir=
    ↪ requester_dir,
                          app_dir=self.app_dir, adapter=self.adapter,
    ↪ hardened=self.hardened)
    key = os.path.realpath(path)
    if key in self._loading:
        raise SLError(SLStatus.CYCLE, f"cyclic dependency through '{
    ↪ name_or_path}'",
                      failed_name=name_or_path)
    handle = self._loaded.get(key)
    if handle is not None:
        self._check_range(handle.metadata, spec)
        handle.refcount += 1
        return handle
    self._loading.append(key)
    try:
        handle = self._load(key, spec)
    finally:
        self._loading.pop()
    self._loaded[key] = handle
    return handle

def _load(self, path: str, spec) -> Handle:
    container = unpack(self._read(path))                    # Schritt 3
    meta = container.metadata
    low, high = self.abi_range
    if not low <= meta.abi_version <= high:
        raise SLError(SLStatus.ABI_MISMATCH, ...)
    payload = container.payload_for(self.platform, self.arch)
    self._check_range(meta, spec)
    deps, native = [], None
    try:
        for dep, dep_spec in meta.dependencies.items():     # Schritt 4
            deps.append(self._open_dependency(meta, dep, dep_spec, path))
        native = self.adapter.map_payload(payload, meta.name)  # Schritt 5
        handle = Handle(meta.name, path, meta, native, self, deps)
        if meta.init:                                       # Schritt 6
            self._run_init(handle)
```

```
        return handle
    except SLError:
        if native is not None:
            self.adapter.unmap(native)
        for dep_handle in deps:
            self._release(dep_handle)
        raise

def _release(self, handle: Handle) -> None:
    handle.refcount -= 1
    if handle.refcount > 0:
        return
    handle.closed = True
    del self._loaded[handle.path]
    if handle.metadata.fini:
        try:
            self.adapter.lookup(handle.native, handle.metadata.fini)()
        except SLError:
            pass
    self.adapter.unmap(handle.native)
    for dep_handle in handle.deps:
        self._release(dep_handle)
```

7.1 Kosten und Charakterisierung des Erfolgs

Im Folgenden gelte Annahme 3.1, der Anfangszustand sei $s_0 = (\emptyset, [\,])$, und alle Hooks und Orakelaufrufe seien deterministisch.

Definition 7.1 (Lokale Bedingung)**.** Eine Datei $f \in V_r$ erfüllt die *lokale Bedingung*, wenn (1) f ein gültiger Container ist (Lemma 5.3), (2) die ABI-Version von M_f im Bereich des Laders liegt, (3) ein Payload für das Target des Laders vorhanden ist und seinen Hash besteht, (4) das Orakel den Payload abbildet, (5) ein deklarierter Init-Hook gefunden wird und 0 liefert.

Satz 7.1 (Aufrufzahlen)**.** Gelingt `open`(r) von s_0 aus, so wird `_load` genau $|V_r|$-mal und `_open` genau $1 + |E_r|$-mal aufgerufen; die Zahl der Dateisystemprüfungen ist höchstens $(1 + |E_r|) \cdot n_{\max}$ mit der Kandidatenschranke $n_{\max}$ aus Satz 6.3. Die Rekursionstiefe ist höchstens $|V_r|$.

Proof. Im Erfolgsfall durchläuft der Code jede Abhängigkeitsdeklaration jedes geladenen Knotens genau einmal und ruft für sie `_open` auf; dazu kommt der äußere Aufruf: $1 + |E_r|$ Aufrufe. Für jeden Knoten $f \in V_r$ wird `_load` beim ersten Erreichen aufgerufen; jedes spätere Erreichen findet f in Λ, weil der Knoten nach seinem Abschluss eingetragen ist und ein Erreichen *während* des Ladens in `CYCLE` enden würde, also nicht im Erfolgsfall liegt. Jeder Aufruf von `_open` führt höchstens $n_{\max}$ Prüfungen aus. Auf dem Keller stehen paarweise verschiedene Schlüssel (sonst `CYCLE`), also höchstens $|V_r|$. □

Lemma 7.1 (Abschlussreihenfolge)**.** Gelingt `open`(r), und ist $f \to g$ eine Kante von G_r, so ist `_load`(g) (oder, falls g schon geladen war, dessen früherer Abschluss) beendet, bevor `_load`(f) beendet ist.

Proof. `_load`(f) ruft in der Schleife `_open` für g auf, bevor es zurückkehrt. Dieser Aufruf kehrt im Erfolgsfall zurück; entweder lädt er g vollständig (`_load`(g) endet vor der Rückkehr) oder findet g in Λ, wo es nach Abschluss eingetragen wurde. □

Satz 7.2 (Charakterisierung des Erfolgs; Zyklen)**.** Seien s_0 und Annahme 3.1 gegeben. `open`(r) gelingt genau dann, wenn

(i) der Graph G_r azyklisch ist,

(ii) jede Datei $f \in V_r$ die lokale Bedingung erfüllt,

(iii) jeder Abhängigkeitsname jeder Datei $f \in V_r$ zu einer Datei $g = \mathrm{res}(n, \mathrm{dirname}(f)) \neq \bot$ aufgelöst wird (im gehärteten Modus: nicht in einem schreibbaren Verzeichnis) und die Version von g den Bereich `s` der Deklaration erfüllt.

Außerdem gilt: Wird `CYCLE` geworfen, so enthält G_r einen Kreis.

Proof. Notwendigkeit. Gelingt `open`(r), so wurde jeder Knoten von V_r geladen und jede Deklaration abgearbeitet. Jede der Bedingungen (2) und (3) entspricht einer `raise`-Anweisung, die nicht ausgeführt wurde. Zu (i): Angenommen G_r hätte einen Kreis $f_1 \to \cdots \to f_j \to f_1$. Nach Lemma 7.1 endet `_load`(f_{i+1}) vor `_load`(f_i) und `_load`(f_1) vor `_load`(f_j), also läge der Abschluss von f_1 echt vor sich selbst; ein Widerspruch. (Ein Kreis der Länge 1 endet unmittelbar in `CYCLE`, weil f_1 auf dem Keller steht.)

Hinlänglichkeit. Da G_r azyklisch und endlich ist, hat jeder Knoten eine endliche Höhe $\eta(f)$, die Länge des längsten von f ausgehenden Weges. Wir zeigen durch Induktion über $\eta(f)$: `_open`(f) gelingt in jedem Zustand, in dem kein Schlüssel aus V_f auf dem Keller steht. Dies ist für $f = r$ im Zustand s_0 erfüllt, und es bleibt beim rekursiven Aufruf erhalten: Stünde ein Knoten $u \in V_g$ auf dem Keller, so wäre u ein Vorfahre von g auf dem Aufrufweg, hätte also einen Weg zu g und läge in V_g; mit dem Weg von g zu u entstünde ein Kreis. Für $\eta(f) = 0$ hat f keine Abhängigkeiten; die lokale Bedingung sichert alle Schritte, f steht nicht auf dem Keller, `_open` gelingt (oder findet f in Λ und besteht die Bereichsprüfung nach (iii)). Für $\eta(f) > 0$ haben alle Abhängigkeiten g kleinere Höhe und erfüllen die Voraussetzung; nach Induktion und (iii) gelingen ihre Aufrufe, danach gelingen Abbilden und Init nach (2).

Zusatz. `CYCLE` wird nur geworfen, wenn der Schlüssel schon auf dem Keller steht. Der Keller ist zu jedem Zeitpunkt ein Weg von r aus in G_r: Jeder Eintrag kommt aus einem `_open`, das aus der Schleife des darunterliegenden `_load` aufgerufen wurde, also über eine Kante. Der Weg vom früheren Eintrag zum jetzigen plus die Kante zurück zum früheren Eintrag bildet einen Kreis. □

Bemerkung zur Reichweite: Welcher Fehler gemeldet wird, wenn mehrere Bedingungen verletzt sind, hängt von der Reihenfolge der Schleife über `meta.dependencies.items()` ab und ist von Satz 7.2 nicht erfasst. Entscheidend ist, dass *Erfolg* reihenfolgeunabhängig charakterisiert ist.

7.2 Operationsvertrag, Referenzzählung und Reihenfolgen

Für die folgenden Sätze gelte zusätzlich:

Annahme 7.1 (Ausnahmeklasse)**.** Alle Aufrufe des Orakels und alle Hooks kehren normal zurück oder werfen eine `SLError`. Es werden keine anderen Ausnahmen geworfen.

Die Annahme ist nicht selbstverständlich; unter Windows löst ein Speicherzugriffsfehler in nativem Code in `ctypes` eine `OSError` aus. Satz 7.5 zeigt, was geschieht, wenn sie verletzt ist.

Lemma 7.2 (Operationsvertrag)**.** Sei $s = (\Lambda, \sigma)$ ein Zustand, in dem kein Schlüssel von σ in $\operatorname{dom}\Lambda$ liegt und $\operatorname{dom}\Lambda$ unter Abhängigkeiten abgeschlossen ist. Ein Aufruf $g = \texttt{_open}(n, \texttt{s}, d)$ verhält sich so:

(a) *Fehler.* Wirft er eine `SLError`, so ist der Zustand danach gleich s (gleiches Λ, gleicher Keller, gleiche Zähler aller bereits vorhandenen Handles), und jedes während des Aufrufs erfolgte `map` ist durch ein `unmap` beantwortet, jedes erfolgreiche Init durch ein Fini.

(b) *Erfolg.* Gelingt er, so ist N die Menge der dabei neu geladenen Schlüssel, $\Lambda' = \Lambda \cup N$, und für jedes Handle h gilt

$$\mathrm{rc}'(h) = \mathrm{rc}(h) + \big|\{(f, i) : f \in N,\ \mathrm{deps}(f)[i] = h\}\big| + [h = g],$$

wobei $\mathrm{rc}(h) = 0$ für $h \in N$ vor dem Aufruf gesetzt ist. Der Keller ist unverändert, $\operatorname{dom}\Lambda'$ ist wieder unter Abhängigkeiten abgeschlossen.

Proof. Induktion über die Zahl der rekursiven `_open`-Aufrufe, die der Aufruf auslöst. *Fall `locate` wirft* oder *Schlüssel auf dem Keller:* Es wurde nichts verändert (a). *Fall Treffer in Λ:* Besteht die Bereichsprüfung nicht, wird vor der Erhöhung geworfen und nichts verändert (a); sonst wird $\mathrm{rc}(g)$ um eins erhöht, $N = \emptyset$ (b). *Fall neuer Schlüssel:* Er wird auf den Keller gelegt und am Ende durch `finally` wieder entfernt, in beiden Ausgängen. Die Schritte vor dem `try` in `_load` (Entpacken, ABI, Payload, Bereich) verändern keinen Zustand; werfen sie, ist (a) erfüllt. Im `try`: Die Aufrufe für die Abhängigkeiten gelingen nach Induktionsvoraussetzung (b) oder lassen den Zustand nach (a) unverändert. Wirft das j-te, so sind die Handles $\mathrm{deps}[1..j-1]$ in `deps` eingetragen; der `except`-Zweig ruft `_release` für jedes von ihnen. Jedes davon nimmt die durch seinen Aufruf erhaltene Referenz zurück: War das Handle schon in Λ, sinkt rc auf den früheren Wert; war es neu, sinkt rc auf 0, es wird aus Λ entfernt, sein Fini-Hook läuft, es wird freigegeben, und rekursiv werden seine Abhängigkeiten zurückgegeben; nach Induktion über die Höhe stellt das genau den früheren Zustand her. Wirft das Abbilden, ist `native` nicht gesetzt; wirft der Init-Hook, wird die native Ressource freigegeben (`unmap`). In beiden Fällen sind danach alle Referenzen der Abhängigkeiten zurückgegeben, das neue Handle wurde nie in Λ eingetragen. Das ist (a); das Fini fehlt für den eigenen gescheiterten Init, weil dieser nicht gelungen ist. Gelingen alle Schritte, wird der neue Schlüssel nach dem Pop in Λ eingetragen; jedes Handle h hat für jeden Eintrag in `deps` eines neu geladenen f eine Referenz erhalten (aus dem Aufruf von `_open`, der h lieferte), und das neue Handle selbst beginnt mit $\mathrm{rc} = 1 = [h = g]$. Abgeschlossenheit folgt, weil jede Abhängigkeit des neuen Knotens ihrerseits in Λ' liegt. □

Definition 7.2 (Externe Referenzen)**.** Für ein Handle h sei $\mathrm{ext}(h)$ die Zahl der erfolgreichen äußeren `open`-Aufrufe, die h geliefert haben und denen noch kein äußeres `close`(h) gegenübersteht.

Satz 7.3 (Invariante der Referenzzählung)**.** Zwischen zwei äußeren Operationen gilt für jedes $h \in \operatorname{dom}\Lambda$

$$\mathrm{rc}(h) = \mathrm{ext}(h) + \big|\{(f,i) : f \in \operatorname{dom}\Lambda,\ \mathrm{deps}(f)[i] = h\}\big|.$$

Insbesondere ist $\mathrm{rc}(h) \geq 1$ für jedes geladene Handle. Werden alle äußeren Referenzen zurückgegeben ($\mathrm{ext} \equiv 0$), so ist danach $\Lambda = \emptyset$, und jedes `map` wurde durch genau ein `unmap` beantwortet.

Proof. Induktion über die Folge der äußeren Operationen; zu Beginn ist Λ leer. Ein äußeres `open` gelingt oder scheitert. Scheitert es, bleibt nach Lemma 7.2(a) alles unverändert. Gelingt es, so liefert Lemma 7.2(b): rc wächst für jedes Handle um die Zahl der neuen Abhängigkeitseinträge, die darauf zeigen (dieselbe Zahl, um die die zweite Summe wächst, weil genau die neu geladenen $f \in N$ hinzukommen), und das gelieferte Handle erhält einen Zähler mehr, dem ext um eins mehr entspricht. Ein äußeres `close`(h) ruft `_release`(h). Bleibt $\mathrm{rc} > 0$, sinkt $\mathrm{rc}(h)$ und $\mathrm{ext}(h)$ um eins. Wird $\mathrm{rc} = 0$, so ist $\mathrm{ext}(h) = 0$ und es gibt keinen Eintrag in einem anderen geladenen Handle auf h; h wird entfernt, und für jeden seiner Abhängigkeitseinträge sinkt der Zähler des Ziels um eins, was der Verkleinerung der zweiten Summe um genau diesen Eintrag entspricht; die Invariante bleibt (rekursiv) erhalten. Sind alle ext null, so folgt $\mathrm{rc}(h)$ = Zahl der Abhängigkeitseinträge auf h. Ein geladenes Handle mit $\mathrm{rc} \geq 1$ besitzt dann einen Vorgänger in Λ. Wegen der Abgeschlossenheit und der Endlichkeit führt die Vorgängerrelation zu einem Handle ohne Vorgänger oder zu einem Kreis; Kreise gibt es nach Satz 7.2 nicht. Ein Handle ohne Vorgänger hätte $\mathrm{rc} = 0$, im Widerspruch zu $\mathrm{rc} \geq 1$. Also ist $\Lambda = \emptyset$. Da jedes erfolgreiche `map` an ein Handle in Λ gebunden ist, das bei seiner Entfernung genau einmal `unmap` auslöst, und gescheiterte Ladevorgänge nach Lemma 7.2(a) ihr `map` zurücknehmen, gilt die Bilanz. □

Satz 7.4 (Reihenfolge von Init und Fini)**.** Sei $f \to g$ eine Kante von G_r in einem erfolgreichen `open`(r). Dann gilt:

(i) $\mathrm{init}(g) \prec \mathrm{init}(f)$; $\mathrm{map}(g) \prec \mathrm{map}(f)$.

(ii) Für jede Datei läuft Init höchstens einmal pro *Ladeinstanz*, also zwischen zwei `map`/`unmap`-Paaren; mehrfaches `open` derselben Datei löst Init nicht erneut aus.

(iii) Werden später alle Referenzen zurückgegeben, so gilt $\mathrm{fini}(f) \prec \mathrm{fini}(g)$ und $\mathrm{unmap}(f) \prec \mathrm{unmap}(g)$.

(iv) Jedes Fini folgt auf ein erfolgreiches Init derselben Ladeinstanz; die Zahl der Fini-Aufrufe ist gleich der Zahl erfolgreicher Init-Aufrufe.

Proof. (i) `map`(f) und Init(f) stehen in `_load`(f) *nach* der Schleife über die Abhängigkeiten. Nach Lemma 7.1 ist `_load`(g) beendet, wenn die Schleife g abgearbeitet hat, oder g war früher fertig; map und Init von g liegen in `_load`(g) vor dessen Ende. (ii) Init steht nur in `_load`, das nur für Schlüssel außerhalb von Λ aufgerufen wird; ein Schlüssel kommt erst nach `unmap` wieder aus Λ. (iii) Nach Satz 7.3 enthält $\mathrm{rc}(g)$ eine Referenz für den Abhängigkeitseintrag in f. In `_release`(f) stehen Fini und `unmap` von f *vor* der Schleife, die g zurückgibt; g erreicht $\mathrm{rc} = 0$ frühestens nach dieser Rückgabe, und Fini und `unmap` von g folgen danach. (iv) Fini steht in `_release` nur für Handles, die vollständig geladen und damit nach `_run_init` (soweit deklariert) in Λ eingetragen wurden; ein gescheiterter Init erreicht Λ nie (Lemma 7.2(a)). □

Satz 7.5 (Ausnahmesicherheit und ihre Grenze)**.** (a) Unter Annahme 7.1 lässt jedes gescheiterte `open` den Zustand unverändert (Lemma 7.2(a)). Der Ablauf der Hooks ist dabei im Allgemeinen *nicht* unverändert: Wurde eine Abhängigkeit neu geladen, bevor ein späterer Schritt scheiterte, so hat deren Init und anschließend deren Fini stattgefunden. (b) Ohne Annahme 7.1 gilt (a) nicht: Wirft der Init-Hook eines Handles eine andere Ausnahme als `SLError`, so bleiben die native Ressource abgebildet und die Referenzen seiner Abhängigkeiten erhöht.

Proof. (a) ist Lemma 7.2(a); das Beispiel für die Hooks: Eine Datei t mit Abhängigkeiten b (gültig, neu) und z (fehlt) lädt b, führt dessen Init aus, scheitert an z und gibt b frei, wobei dessen Fini läuft. (b) Der `except`-Zweig in `_load` fängt nur `SLError`. Wirft `_run_init` etwa eine `OSError`, verlässt sie `_load`, ohne dass `unmap` oder `_release` der Abhängigkeiten ausgeführt wurden. Der Schlüssel wird zwar im `finally` vom Keller genommen, aber das Handle wurde nie eingetragen und seine Referenzen nie zurückgegeben; die Invariante aus Satz 7.3 gilt dann nicht mehr. Dasselbe gilt für den Fini-Hook in `_release`: Eine andere Ausnahme verlässt die Funktion, nachdem das Handle aus Λ entfernt wurde und bevor `unmap` und die Rückgabe der Abhängigkeiten stattfinden. □

7.3 Instanzen, Export und Nutzung

Satz 7.6 (Instanzidentität)**.** Zwei erfolgreiche `open`-Aufrufe liefern genau dann dasselbe Handle-Objekt, wenn die zu ihnen gefundenen Dateien denselben Schlüssel (`realpath`) haben und das Handle dazwischen nicht freigegeben wurde. Insbesondere ist die Instanz durch die *Datei*, nicht durch den logischen Namen bestimmt: Ein Name kann bei verschiedener Konfiguration oder verschiedenem Aufruferverzeichnis zu verschiedenen Dateien und damit zu verschiedenen gleichzeitigen Instanzen führen.

Proof. Der Schlüssel in Λ ist `os.path.realpath(path)`; `_open` liefert bei Treffer in Λ das gespeicherte Objekt. Verschiedene Pfade auf dieselbe Datei (Symlinks) haben denselben `realpath`. Gegenbeispiel zur zweiten Aussage: Liegt `base.sl` in zwei Verzeichnissen d_1, d_2 und sind d_1, d_2 jeweils das Verzeichnis von zwei Bibliotheken a, b, die `base` deklarieren, und ist `base` in keinem Pfad vor dem Aufruferverzeichnis zu finden, dann lösen a und b `base` zu verschiedenen Dateien auf; beide Instanzen sind gleichzeitig geladen. Der Entwurf lässt offen, ob höchstens eine Instanz je Name erlaubt sein soll. □

Satz 7.7 (Exportdurchsetzung)**.** Sei $N(h)$ die Menge der Namen, für die `Loader.symbol`(h, n) ohne Fehler zurückkehrt. Dann gilt $N(h) \subseteq X_h \cap S_h$, wobei X_h die Exportliste in M_h und S_h die im nativen Payload auflösbaren Symbole sind. Gilt $X_h \subseteq S_h$, so ist $N(h) = X_h$. Die Durchsetzung ist eine Eigenschaft der API, keine Sicherheitsgrenze: Der Zugriff auf `handle.native.cdll` umgeht sie.

Proof. `symbol` wirft `SYMBOL_NOT_FOUND`, wenn $n \notin X_h$, und reicht sonst an `adapter.lookup` weiter, das für $n \notin S_h$ ebenfalls `SYMBOL_NOT_FOUND` wirft. Die Gleichheit folgt aus beiden Fällen. Das Feld `native` des öffentlichen Handles enthält das `ctypes`-Objekt, dessen Attributzugriff alle nativen Symbole auflöst. Die Hooks werden unabhängig von X_h über `adapter.lookup` aufgerufen; sie müssen nicht exportiert sein. □

Satz 7.8 (Nutzungssicherheit bei korrekter Klammerung)**.** Jeder Aufruf von `symbol` und jede Verwendung der zurückgegebenen Funktion durch einen Aufrufer liege zeitlich innerhalb des Intervalls zwischen dem `open`, das ihm das Handle lieferte, und dem zugehörigen `close` desselben Aufrufers, und kein Aufrufer rufe `close` zweimal für ein `open` auf. Dann ist das Handle während der Verwendung nicht freigegeben und nicht abgebildet.

Proof. Zu jedem Zeitpunkt der Verwendung ist $\mathrm{ext}(h) \geq 1$, denn das `open` des Aufrufers ist noch nicht durch sein `close` beantwortet, und es wird nur einmal beantwortet. Nach Satz 7.3 ist $\mathrm{rc}(h) \geq \mathrm{ext}(h) \geq 1$, also $h \in \Lambda$ und `closed` = falsch; das native Handle wird erst bei $\mathrm{rc} = 0$ freigegeben. `open` und `close` sind durch die Sperre serialisiert, so dass die Zähler konsistent bleiben. □

Die Voraussetzung des Satzes ist wesentlich. Alle Aufrufer, die denselben Schlüssel öffnen, teilen sich *ein* Handle-Objekt; ein doppeltes `close` eines Aufrufers nimmt deshalb die Referenz eines anderen zurück, und ohne die Klammerung kann `symbol` zwischen der Prüfung auf `closed` und dem Auflösen mit einem `close` eines anderen Threads zusammentreffen, weil `symbol` und `info` die Sperre nicht nehmen (Befund 14.13). Die Klasse `binding.Library` verhindert das doppelte `close` durch Setzen ihres Feldes auf `None`.

7.4 Nebenläufigkeit

Satz 7.9 (Deadlock durch die globale Sperre)**.** Es gibt ein Programm mit zwei Threads und einem gemeinsamen `Loader`, das nicht terminiert, obwohl keine der Bibliotheken zyklisch abhängt und keine Operation einen Fehler meldet.

Proof. `Loader.open` umschließt `_open`, also auch den Init-Hook, mit `with self._lock` (Zeile 47). Sei A eine Bibliothek, deren Init-Hook wartet, bis ein zweiter Thread ein Ereignis ε gesetzt hat. Thread 1 ruft `open`(A) auf; er hält die Sperre und blockiert im Hook. Thread 2 ruft zuerst `open`(B) für eine beliebige Bibliothek B auf und will danach ε setzen. `open`(B) verlangt die Sperre, die Thread 1 hält, und blockiert. Der Wartegraph hat die Kanten Thread 1 $\to$ Thread 2 (wartet auf ε) und Thread 2 $\to$ Thread 1 (wartet auf die Sperre): einen Kreis. Keiner der beiden kann fortfahren. □

Das verletzt die Zusage des Entwurfs (Abschnitt 5.7), Initialisierung laufe nie unter einer Sperre, die Nutzercode betreten kann; die README nennt die Sperre ausdrücklich (*reentrant*, hält während des Initialisierens). Die Wiedereintrittsfähigkeit löst nur den Fall *desselben* Threads: Ein `open` derselben Bibliothek im Init-Hook trifft auf den Schlüssel im Keller und endet in `CYCLE` (Satz 7.2), nicht in einem Selbst-Deadlock. Abschnitt 16.3 des Ausblicks skizziert einen Entwurf mit Sperren je Schlüssel.

Kapitel 8

Plattformadapter und Schichtung

8.1 Der Adapter als Orakel

Der Adapter ist eine abstrakte Klasse mit vier Operationen. Listing 8.1 zeigt die gemeinsame Basisimplementierung, die den Payload in ein frisches temporäres Verzeichnis schreibt und mit `ctypes` öffnet.

Listing 8.1: Abbilden und Freigeben (`adapters.py`, Zeilen 39–63 und 66–96, gekürzt).

```
def map_payload(self, payload: bytes, name: str) -> NativeLib:
    tmpdir = tempfile.mkdtemp(prefix="sl-")
    path = os.path.join(tmpdir, name + self.native_suffix)
    try:
        with open(path, "wb") as handle:
            handle.write(payload)
        return NativeLib(self._open(path), path, tmpdir)
    except OSError:
        shutil.rmtree(tmpdir, ignore_errors=True)
        raise

def unmap(self, native: NativeLib) -> None:
    self._release(native.cdll)
    shutil.rmtree(native.tmpdir, ignore_errors=True)

class LinuxAdapter(_PosixAdapter):    # system_dirs: /usr/local/lib/sl, /usr/lib/sl
class DarwinAdapter(_PosixAdapter):   # system_dirs: /usr/local/lib/sl, /opt/homebrew/lib/sl
class WindowsAdapter(PlatformAdapter):  # system_dirs: ProgramData\sl, ProgramFiles\sl
    def _release(self, cdll) -> None:
        _ctypes.FreeLibrary(cdll._handle)     # POSIX: _ctypes.dlclose(cdll._handle)
```

Lemma 8.1 (Kollisionsfreie Abbildung)**.** Zwei Aufrufe von `map_payload` legen ihre Dateien in verschiedenen Verzeichnissen ab, auch bei gleichem Namen ν. Die Datei wird freigegeben, bevor ihr Verzeichnis entfernt wird.

Proof. `tempfile.mkdtemp` erzeugt ein bisher nicht vorhandenes Verzeichnis (nach der Dokumentation atomar, mit eindeutigem Namen). `unmap` ruft zuerst `_release` auf (`dlclose` bzw. `FreeLibrary`) und danach `rmtree`; dies ist unter Windows wesentlich, weil geladene DLLs nicht gelöscht werden können. □

Bemerkung 8.1 (Grenzen der Freigabe)**.** `dlclose` gibt nach POSIX nur *einen Verweis* auf das Objekt frei; ob das Abbild tatsächlich entladen wird, ist nicht zugesichert (etwa bei Objekten, die sich als nicht entladbar markiert haben). Die Referenzzählung des Laders (Satz 7.3) zählt Verweise des Laders, nicht Abbilder im Speicher. Die Abbildung über eine temporäre Datei setzt außerdem voraus, dass das Verzeichnis für temporäre Dateien Ausführung erlaubt; auf Systemen, die es ohne Ausführungsrecht einhängen, kann das Abbilden scheitern **[nicht geprüft]**.

8.2 Der Importgraph und die Schichten

Die 14 Module von `sl` bilden einen Importgraphen. Er ist azyklisch und ordnet die Module in Ebenen (Abbildung 8.1).

Satz 8.1 (Schichtenordnung und Laufzeitabschluss)**.** Der Importgraph von `sl` ist azyklisch. Die Abbildung λ mit $\lambda(\texttt{errors}) = 0$; $\lambda(\texttt{platform_info}) = \lambda(\texttt{search}) = 1$; $\lambda(\texttt{metadata}) = \lambda(\texttt{adapters}) = 2$; $\lambda(\texttt{container}) = 3$; $\lambda(\texttt{loader}) = \lambda(\texttt{build}) = 4$; $\lambda(\texttt{capi}) = \lambda(\texttt{codegen}) = 5$; $\lambda(\texttt{cli}) = \lambda(\texttt{binding}) = 6$; $\lambda(\texttt{__init__}) = \lambda(\texttt{__main__}) = 7$ erfüllt $\lambda(u) > \lambda(v)$ für jeden Import $u \to v$. Der Abschluss von `loader` unter Importen ist $\{\texttt{loader}, \texttt{adapters}, \texttt{container}, \texttt{errors}, \texttt{metadata}, \texttt{platform_info}, \texttt{search}\}$; er enthält weder `build`, `codegen`, `cli`, `capi` noch `binding`.

Proof. Die Importe sind den `import`-Zeilen der Module zu entnehmen: `platform_info`→`errors`; `search`→`errors`; `metadata`→`errors`, `platform_info`; `adapters`→`errors`, `platform_info`; `container`→`errors`, `metadata`; `loader`→`adapters`, `container`, `errors`, `metadata`, `platform_info`, `search`; `build`→`container`, `errors`, `metadata`, `platform_info`; `capi`→`errors`, `loader`; `codegen`→`build`, `metadata`; `binding`→`capi`; `cli`→`build`, `codegen`, `container`, `errors`, `metadata`; `__init__`→`adapters`, `binding`, `capi`, `container`, `errors`, `loader`, `metadata`; `__main__`→`cli`. Für jeden dieser Importe $u \to v$ ist $\lambda(u) = 1 + \max \lambda(v)$ über die Ziele v von u (nachzurechnen: `loader`: $1 + \max(2,3,0,2,1,1) = 4$; `build`: $1 + \max(3,0,2,1) = 4$; `capi`: $1 + 4 = 5$; `codegen`: $1 + \max(4,2) = 5$; `binding`: $1 + 5 = 6$; `cli`: $1 + \max(4,5,3,0,2) = 6$; `__init__`: $1 + \max(2,6,5,3,0,4,2) = 7$; `__main__`: $1 + 6 = 7$). Eine streng fallende Ebenenfunktion schließt Kreise aus. Der Abschluss von `loader` ergibt sich durch Vereinigung der Ziele: `adapters`, `container`, `errors`, `metadata`, `platform_info`, `search` und deren Ziele, die in dieser Menge liegen. □

Praktisch heißt das: Wer nur laden will, braucht weder Compiler-Aufrufe noch Codegenerierung noch Kommandozeile; der Laufzeitpfad ist ein klar abgegrenzter Teil des Pakets. Das Paket hat keine Laufzeitabhängigkeiten außerhalb der Standardbibliothek (`dependencies = []`).

8.3 Wie weit trägt die Schichtung?

Satz 8.2 (Adapter-Parametrizität mit Ausnahmen)**.** Die Beobachtung einer Operationsfolge des Laders hängt vom Betriebssystem nur ab über

(1) die vier Operationen des Adapters (`system_dirs`, `map_payload`, `lookup`, `unmap`),

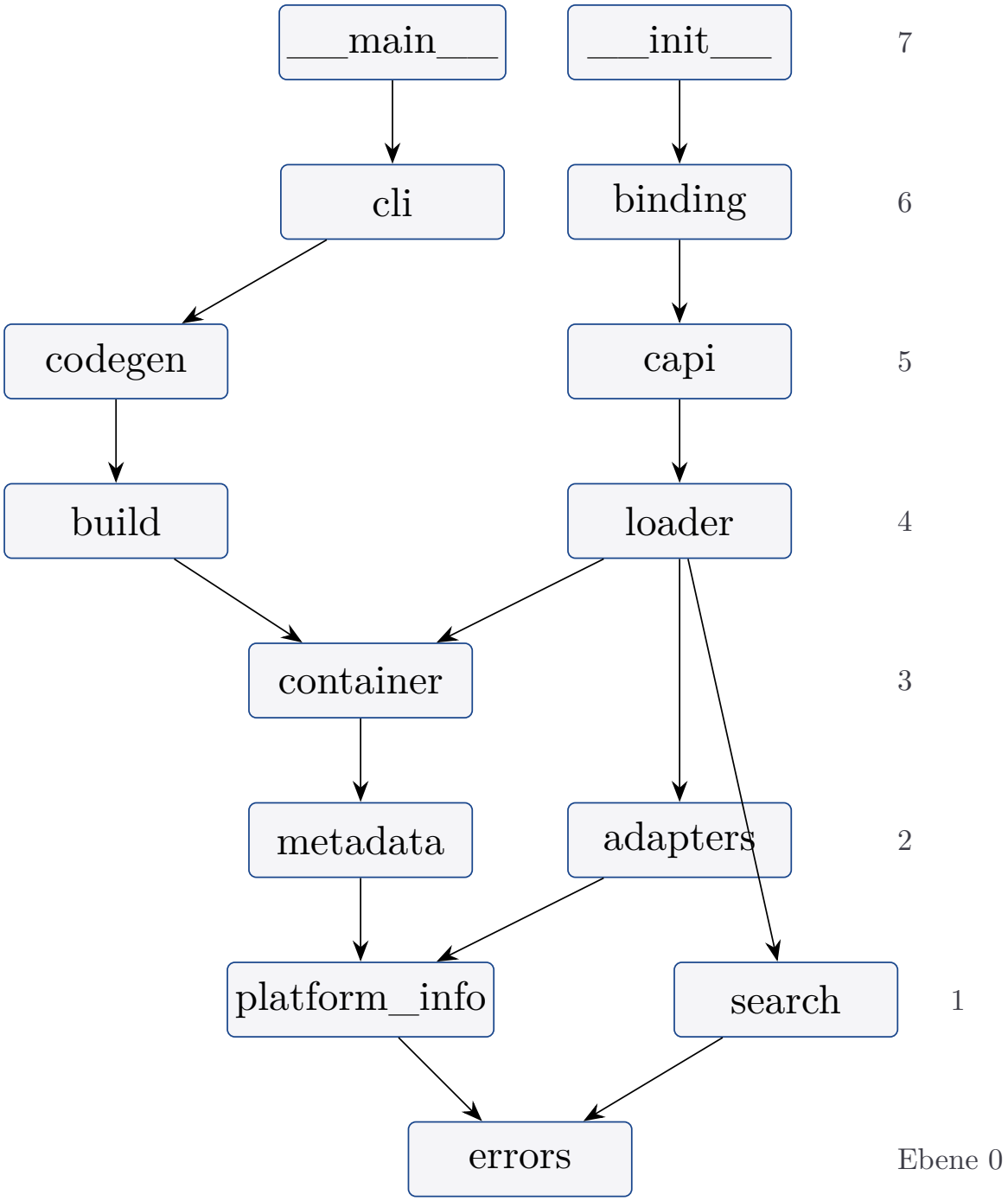


Abbildung 8.1: Importgraph von `sl` (nur direkte Importe innerhalb des Pakets, jeweils zum höchsten Ziel gezeichnet; `loader` importiert zusätzlich `errors`, `metadata` und `platform_info`, `build` importiert `errors`, `metadata` und `platform_info`, `cli` importiert `build`, `codegen`, `container`, `errors`, `metadata`; `__init__` importiert `adapters`, `binding`, `capi`, `container`, `errors`, `loader`, `metadata`).

(2) die Pfadfunktionen der Standardbibliothek `os.path.isabs`, `join`, `realpath`, `dirname`, `abspath`, `isfile`, `basename`,

(3) die Verzweigung `os.name == "nt"` in `search._world_writable`,

(4) die Vorgabewerte `current_platform()` und `current_arch()` bei der

Konstruktion eines `Loader` ohne ausdrückliche Angabe.

Mit Fake-Adapter, expliziter Plattform und Architektur und einer POSIX-Pfadumgebung ist die Beobachtung eine Funktion von $(E, R, A, h, \mathcal{F}, \mathcal{O})$.

Proof. Wir durchsuchen die Module des Laufzeitpfads (Satz 8.1) nach Zugriffen auf Plattformzustand. `loader.py`: `os.environ` nur als Vorgabe für `env`; `os.path.realpath` und `dirname`; die Vorgaben aus `platform_info`; Orakelaufrufe nur über `self.adapter` an den Stellen `system_dirs` (über `locate`), `map_payload`, `lookup` (Symbole, Init, Fini) und `unmap`. `search.py`: `os.path.isabs`, `join`, `isfile`, `abspath`, `dirname`, `basename`, `os.stat`, `os.name`. `container.py` und `metadata.py` greifen auf keine Plattformfunktion zu; `platform_info.py` liest `sys.platform` und `platform.machine()` nur in den Vorgabewerten. Alle weiteren Aufrufe sind reine Funktionen ihrer Argumente. Die Zusatzaussage folgt, weil unter diesen Annahmen (1), (3) und (4) entfallen und (2) unter POSIX-Pfaden eine feste Funktion ist. □

Die Tabelle 8.1 stellt die Ausnahmen den Orten im Quelltext gegenüber. Der Entwurf sagt, *alles* Plattformspezifische liege im Adapter; die Referenzimplementierung hält dies für die Funktionen des Systemladers ein, nicht für Pfadsyntax und Rechteprüfung. Das ist kein Mangel an Sorgfalt, sondern folgt aus der Wahl, Pfade als Zeichenketten zu behandeln; der Ausblick (Abschnitt 16.6) beschreibt eine saubere Trennung.

Ort	Plattformzugriff	Folge
`search.py`:84, 94, 137, 138, 143	`os.path.isabs`, `join`, `isfile`, `dirname`, `abspath`	Pfadsyntax (`/` oder `\`) und Absolutheit hängen vom System ab; derselbe `SL_PATH`-Eintrag `C:\x` ist auf Linux relativ und wird verworfen
`search.py`:120–127	`os.name`, `os.stat`	Rechteprüfung nur auf POSIX; unter Windows Testnamen-Sonderfall (B3)
`loader.py`:87, 146	`os.path.realpath`, `dirname`	Instanzidentität (Satz 7.6) hängt von der Symlink-Auflösung des Systems ab
`loader.py`:33–35	`current_platform`, `current_arch`	Vorgaben des Laders
`adapters.py`:43, 44, 90	`tempfile.mkdtemp`, `os.path.join`	Ort der Materialisierung hängt von `TMPDIR`/`TEMP` ab

Tabelle 8.1: Plattformzugriffe außerhalb der vier Adapteroperationen (Zeilennummern des hochgeladenen Standes).

Bemerkung 8.2 (Was die Fake-Adapter-Tests beweisen)**.** Die Tests des Lader-Kerns laufen mit einem `FakeAdapter`, der keine native Bibliothek lädt (`tests/conftest.py`). Nach Satz 8.2 ist die Beobachtung des Lader-Kerns unter diesen Bedingungen unabhängig vom Betriebssystem. Ein Lauf dieser Tests auf einem System ist damit ein Lauf der Lader-Logik für *alle* Systeme, soweit nicht die Ausnahmen (2)–(4) berührt werden. Die Tests der echten Adapter für Windows und macOS laufen über Nachbildungen (README), nicht auf diesen Systemen.

Teil III

Umfeld des Verfahrens

Kapitel 9

Codegenerierung und Build

Neben dem Lader erzeugt `sl` aus einem Metadatensatz die Schichten, die ein Projekt für die Nutzung auf mehreren Plattformen braucht: C-Header, C-Quelle, ein CMake-Modul, ein Python-Binding und die Build-Matrix (`codegen.py`), und es kompiliert und packt den Container (`build.py`).

9.1 Ein durchgängiges Beispiel

Die README zeigt die Metadaten der Bibliothek `wsat`:

Listing 9.1: Metadaten `wsat.json` (README, Schnellstart).

```
{
  "name": "wsat", "version": "1.2.0", "abi_version": 1,
  "targets": [{"platform": "linux", "arch": "x86_64"}],
  "dependencies": {},
  "exports": ["wsat_solve"],
  "signatures": {"wsat_solve": "int␣wsat_solve(const␣int␣*xs,␣unsigned␣int␣
    ↪ n)"},
  "init": "wsat_init", "fini": "wsat_fini"
}
```

Aus ihnen berechnet `generate_header` den Header. Die folgende Ausgabe wurde durch Lesen des Generators abgeleitet, nicht durch einen Lauf erzeugt:

Listing 9.2: Abgeleitete Ausgabe von `generate_header` für Listing 9.1.

```
/* Generated by sl for 'wsat' - do not edit. */
#ifndef WSAT_SL_H
#define WSAT_SL_H

#ifndef SL_EXPORT
#  if defined(_WIN32)
#    define SL_EXPORT __declspec(dllexport)
#  else
#    define SL_EXPORT __attribute__((visibility("default")))
#  endif
#endif

#ifdef __cplusplus
extern "C" {
#endif

```

```
#define WSAT_ABI_VERSION 1

SL_EXPORT int wsat_abi_version(void);
SL_EXPORT const char *wsat_sl_metadata(void);
SL_EXPORT int wsat_init(void);
SL_EXPORT int wsat_fini(void);
SL_EXPORT int wsat_solve(const int *xs, unsigned int n);

#ifdef __cplusplus
}
#endif

#endif /* WSAT_SL_H */
```

Das Python-Binding enthält für dieselben Metadaten die Tabelle `SIGNATURES = {"wsat_solve": ([ctypes.c_void_p, ctypes.c_uint], ctypes.c_int)}`, denn `const int *` ist ein Zeiger, der nicht `char *` ist, und wird auf `c_void_p` abgebildet (Satz 9.2).

9.2 Eine Quelle der Wahrheit

Satz 9.1 (Konsistenz der erzeugten Schichten)**.** Jeder Generator ist eine reine Funktion des Metadatensatzes M: Er liest keine anderen Eingaben und erzeugt für gleiche M gleiche Ausgaben. Der Header deklariert genau die Menge

$$\{\nu\texttt{_abi_version},\ \nu\texttt{_sl_metadata}\} \cup \{\iota, \varphi\} \setminus \{\texttt{None}\} \cup X,$$

das Python-Binding enthält einen Eintrag je Element von X in der Reihenfolge der Exportliste, und die Build-Matrix hat genau $|T|$ Einträge, einen je Target in der Ordnung von T.

Proof. Die Funktionen `generate_header`, `generate_c_source`, `generate_cmake`, `generate_python_binding` und `build_matrix` haben als einzigen nichtkonstanten Parameter `meta` (und `compiler` mit Vorgabe `"cc"`); sie verwenden `json.dumps(…, sort_keys=True)` und Schleifen über `meta.exports` und `meta.targets`, deren Reihenfolge feststeht. Die Menge der Deklarationen ist durch die Zeilen in `generate_header` gegeben: zwei feste Symbole, je ein Hook, falls nicht `None`, und eine Zeile je Export. Der Binding-Generator iteriert über `meta.exports` und baut je Symbol eine Zeile; die Matrix ist eine Listenkomprehension über `metadata.targets`. □

Lemma 9.1 (Eingebettete Metadaten)**.** Die vom C-Quelltext zurückgegebene Zeichenkette `<name>_sl_metadata()` ist ein gültiges C-Zeichenkettenliteral und gleich `json.dumps(M.to_dict(), sort_keys=True)`. Sie ist byteweise gleich der Metadaten-Kodierung im Container genau dann, wenn die Targettupel von M und der beim Packen verwendeten Metadaten gleich sind.

Proof. `json.dumps` mit der Vorgabe `ensure_ascii=True` erzeugt nur druckbare ASCII-Zeichen; Steuerzeichen, Zeilenumbrüche und Nicht-ASCII-Zeichen werden als `\n`, `\uXXXX` usw. geschrieben. `_c_string` verdoppelt Rückstriche und maskiert Anführungszeichen; das Ergebnis ist ein gültiges Literal, das als Zeichenkette den ursprünglichen Text ergibt. Der Container verwendet denselben Ausdruck (`pack`), jedoch mit den Metadaten, die an `pack` übergeben werden; `pack_files` übergibt

`metadata.with_targets(payloads)`, dessen Targets aufsteigend sortiert sind und der Menge der Payloads entsprechen. Alle anderen Felder stimmen überein; ein Unterschied kann deshalb nur im Feld `targets` auftreten, und er tritt auf, wenn die Tupel verschieden sind (andere Reihenfolge oder andere Menge). □

Der Lader liest die eingebettete Zeichenkette nicht; sie dient der Selbstauskunft des Payloads. Die Folge ist, dass das Feld `targets` der Metadaten bei `sl pack` und `sl build` nur beratend ist: Das Targetfeld des Containers wird aus den tatsächlich vorhandenen Payloads gebildet.

9.3 Signaturen und Typabbildung

Definition 9.1 (Signaturfragment)**.** Das *Fragment* $\mathcal{F}_{\text{sig}}$ besteht aus Zeichenketten der Form $R\ n\,(\,a_1, \ldots, a_k\,)\,[;]$ oder $R\ n\,(\texttt{void})\,[;]$, wobei R und die a_i jeweils aus einem Basistyp der Tabelle $\mathcal{T}$ (22 Einträge: `void`, `char`, `short`, `int`, `unsigned`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `float`, `double`, `size_t`, `bool` und die acht Typen `int8_t` bis `uint64_t`), optionalem `const` und beliebig vielen `*` bestehen, jedes a_i zusätzlich aus einem optionalen Parameternamen, der ein Bezeichner und kein Typwort ist; Klammern kommen sonst nicht vor.

Lemma 9.2 (Korrektheit von `parse_signature`)**.** Für $s \in \mathcal{F}_{\text{sig}}$ mit Funktionsname n liefert `parse_signature`(s, n) den Rückgabetyp und die Liste der Argumenttypen ohne Parameternamen, in der Reihenfolge der Deklaration; für (`void`) ist die Liste leer. Für s mit anderem Namen wird `ValueError` geworfen.

Proof. Der reguläre Ausdruck `^(?P<ret>.*?[\s*])(?P<n>[A-Za-z_]\w*)\s*\((?P<args>.*)\)\s*;?\s*$` verlangt, dass der Name unmittelbar vor der ersten öffnenden Klammer steht. In $\mathcal{F}_{\text{sig}}$ kommt vor ihr keine Klammer vor, also ist `ret` der Text bis vor den letzten Bezeichner und der Name der Bezeichner; `args` reicht bis zur einzigen schließenden Klammer. Ein abweichender Name löst die Prüfung `match.group("n") != symbol` aus. Die Argumente werden an Kommata getrennt, was ohne Klammern und ohne Kommata in Typen eindeutig ist. In jedem Argument trennt der Code `*` durch Leerzeichen ab und zerlegt in Wörter. Das letzte Wort wird als Parametername entfernt, wenn es nicht `*` und kein Typwort ist und mehr als ein Wort vorhanden ist. Für einen Typ aus $\mathcal{T}$ ohne Namen ist das letzte Wort ein Typwort oder ein `*`, es wird nicht entfernt; mit Namen ist es der Name (kein Typwort nach Voraussetzung) und wird entfernt. Ein einzelnes Wort wird nie entfernt. Ist die Argumentliste genau `void`, entsteht die leere Liste. □

Annahme 9.1 (Größen in `ctypes`)**.** Für jedes $\tau \in \mathcal{T}$ stimmt der `ctypes`-Typ $\kappa(\tau)$ in Größe, Vorzeichenbehaftung und Argumentübergabe mit dem C-Typ τ der Zielplattform überein (so definiert die `ctypes`-Dokumentation etwa `c_long` als C-`long` der Plattform), und Zeiger haben die Größe von `void *`.

Satz 9.2 (Typabbildung)**.** Unter Annahme 9.1 gilt für `ctype_expr`: Ein Typ mit mindestens einem `*` wird, nach Entfernen von `const`, genau dann auf `c_char_p` abgebildet, wenn seine Wörter genau `char`, `*` sind; sonst auf `c_void_p`. Ein Typ ohne `*` wird auf den Eintrag der Tabelle abgebildet, wenn er in $\mathcal{T}$ liegt, sonst wird `ValueError` geworfen. Insbesondere werden `unsigned char`, `unsigned short`, `long int` und `long double` abgelehnt, und die Plattformabhängigkeit von `long` (32 Bit unter Windows, 64 Bit unter LP64-Systemen) wird von `ctypes` aufgefangen, nicht vom Generator.

Proof. Der erste Teil ist die Fallunterscheidung im Code (nach `if "*" in tokens`). Der zweite Teil ist die Suche in `_SCALARS`; die genannten Typen stehen nicht in der Tabelle. Dass `c_long` die Plattformgröße trägt, ist Annahme 9.1; der Generator verwendet für `long` das Symbol, nicht eine feste Breite. □

Bemerkung 9.1. Die Abbildung *jedes* `char *`-Rückgabewerts auf `c_char_p` macht aus dem Zeiger eine Python-Bytefolge (Kopie bis zum Nullbyte); der Zeigerwert geht verloren. Gibt die Funktion Speicher mit Eigentum an den Aufrufer zurück, kann er nicht freigegeben werden. Der Entwurf nennt dazu die Funktion `wsat_free`; eine Notation für Besitzverhältnisse in den Signaturen fehlt (Ausblick).

9.4 Build-Orchestrierung

Satz 9.3 (Ergebnis des Builds)**.** `build_sl`$(M, \text{sources}, o, \ldots, \text{extra})$ erzeugt einen Container, dessen Targetmenge gleich $\text{dom}(\text{extra}) \cup \{h\}$ ist, wobei h das Target des Hosts ist; der Payload zu h ist die frisch kompilierte Bibliothek, auch wenn `extra` einen Eintrag für h enthält. Das Feld `targets` von M geht nicht in das Ergebnis ein.

Proof. In `build_sl` wird `files = {**extra, host: lib}` gebildet; bei gleichem Schlüssel gewinnt der spätere Eintrag, also `lib`. `pack_files` ruft `pack(metadata.with_targets(payloads), payloads)`, ersetzt also die Targetliste der Metadaten durch die Schlüssel von `payloads`. □

Zwei Einschränkungen des Builds sind für die Beispielprojekte erheblich. Erstens verwendet `toolchain_flags` für Windows nur `-shared`, also die Form von GCC und Clang; `find_compiler` nennt aber auch `cl` als Kandidaten, dessen Aufrufform anders ist (`/LD`, `/Fe`). Zweitens können Compiler, Sprachstandard und Bibliotheken nicht angegeben werden: `-I` und `-D` sind die einzigen Zusätze, und `cc` linkt keine C++-Laufzeit. Ein C++17-Projekt wie `github.com/naphets86/csubgraph` lässt sich damit nicht ohne Umweg bauen (Befund 14.9). Das ist keine Schwäche des Formats, sondern eine Lücke des Werkzeugs, die der Ausblick schließt.

Kapitel 10

Versionen: Konvexität und Komplexität

Die Metadaten tragen Versionsbereiche für Abhängigkeiten. Dieses Kapitel beantwortet zwei Fragen: Welche Mengen von Versionen lassen sich ausdrücken? Und warum wählt der Lader nicht unter mehreren Kandidaten aus?

10.1 Die Bereichssprache

Satz 10.1 (Ordnung und Konvexität)**.** (a) $(\mathbb{N}^3, <)$ mit der lexikographischen Ordnung ist total, und `satisfies`(v, s) wird in Zeit $O(|\mathsf{s}|)$ entschieden.

(b) Für jeden Bereich `s` ist $S(\mathsf{s})$ *konvex*: Aus $u < v < w$ und $u, w \in S(\mathsf{s})$ folgt $v \in S(\mathsf{s})$.

(c) Die Bereichssprache ist unter Konjunktion abgeschlossen (Aneinanderreihen mit Komma), nicht aber unter Disjunktion: Es gibt keinen Bereich `s` mit $S(\mathsf{s}) = S(\texttt{<1.0.0}) \cup S(\texttt{>=2.0.0})$.

(d) Es gibt unerfüllbare Bereiche mit $S(\mathsf{s}) = \emptyset$, etwa `>=2.0.0,<1.0.0`, und `Metadata` akzeptiert sie.

Proof. (a) Die lexikographische Ordnung auf $\mathbb{N}^3$ ist total: Für $v \neq w$ gibt es eine erste Komponente mit Unterschied, und dort ist eine kleiner. Die Prüfung vergleicht v mit jeder Schranke einmal, jeder Vergleich ist $O(1)$. (b) Jede einzelne Einschränkung hat eine konvexe Lösungsmenge: $\{v \geq b\}$ und $\{v > b\}$ enthalten mit u auch alles Größere, $\{v \leq b\}$ und $\{v < b\}$ mit w auch alles Kleinere, $\{v = b\}$ ist einelementig; in jedem Fall liegt v zwischen zwei Elementen in der Menge. Der Durchschnitt endlich vieler konvexer Mengen ist konvex: Liegen u, w im Durchschnitt, so liegt v in jeder der Mengen. (c) Konjunktion: $S(\mathsf{s}_1, \mathsf{s}_2) = S(\mathsf{s}_1) \cap S(\mathsf{s}_2)$ nach Definition. Disjunktion: $u = (0, 5, 0)$ und $w = (2, 0, 0)$ liegen in der Vereinigung, $v = (1, 0, 0)$ liegt zwischen ihnen und nicht darin; nach (b) kann die Vereinigung kein $S(\mathsf{s})$ sein. (d) Für $v \geq (2, 0, 0)$ und $v < (1, 0, 0)$ gibt es kein v. Die Prüfung in `Metadata.__post_init__` ruft nur `_parse_range` auf, das Syntax, nicht Erfüllbarkeit prüft. □

Zwei Folgerungen: Eine Abhängigkeit mit unerfüllbarem Bereich führt bei jedem `open` zu `DEPENDENCY_MISSING`; eine Prüfung beim Erzeugen der Metadaten wäre billig (Ausblick). Und der Entwurf-Beispielbereich `>=1.2,<2` (Anhang A) ist in

der Implementierung syntaktisch ungültig, weil sie dreiteilige Versionen verlangt (Befund 14.14).

10.2 Auswahl unter mehreren Kandidaten ist NP-vollständig

Der Lader nimmt den ersten Treffer (Satz 6.3) und prüft nur ihn. Man könnte meinen, das sei eine Bequemlichkeit. Der folgende Satz zeigt, dass es eine Notwendigkeit ist: Würde der Lader unter *allen* gefundenen Dateien eine gemeinsame, verträgliche Auswahl suchen, wäre das Problem NP-vollständig.

Definition 10.1 (Auswahlproblem DepSelect)**.** Eine Instanz besteht aus einer endlichen Menge von Paketen Π; jedem Paket π ist eine endliche Menge $V(\pi) \subset \mathbb{N}^3$ von Versionen zugeordnet; jede Version (π, v) trägt eine endliche Liste von Anforderungen (π', s) mit einem Bereich s; und es gibt eine Wurzel mit einer Liste von Anforderungen. Eine *Auswahl* ist eine partielle Abbildung c von Paketen auf Versionen, die jedem ausgewählten Paket höchstens eine Version zuordnet. Sie ist *zulässig*, wenn für jede Anforderung (π', s) der Wurzel und jeder ausgewählten Version $(\pi, c(\pi))$ gilt: π' ist ausgewählt und $c(\pi') \models \mathsf{s}$. Die Frage lautet, ob eine zulässige Auswahl existiert.

Satz 10.2 (DepSelect ist NP-vollständig)**.** DepSelect ist NP-vollständig, auch wenn jedes Paket höchstens drei Versionen und jede Version höchstens eine Anforderung hat und alle Bereiche die Form `>=2.0.0`, `<2.0.0` oder `*` haben.

Proof. In NP. Eine Auswahl hat polynomielle Größe; ihre Zulässigkeit prüft man mit einem Durchlauf über die Anforderungen und je einer Bereichsprüfung (Satz 10.1(a)).

NP-schwer. Wir reduzieren 3-SAT (Karp [2]). Sei $\varphi = C_1 \wedge \cdots \wedge C_m$ über $x_1, \ldots, x_n$, jede Klausel mit genau drei Literalen (kürzere Klauseln durch Wiederholen eines Literals aufgefüllt). Wir bilden folgende Instanz.

- Pakete $X_1, \ldots, X_n$ mit den Versionen 1.0.0 („falsch“) und 2.0.0 („wahr“), ohne Anforderungen.
- Pakete $K_1, \ldots, K_m$ mit den Versionen 1.0.0, 2.0.0, 3.0.0. Ist das t-te Literal von C_j gleich x_i, so fordert die Version $t.0.0$ von K_j das Paket X_i mit `>=2.0.0`; ist es $\neg x_i$, so fordert sie X_i mit `<2.0.0`.
- Die Wurzel fordert jedes K_j mit `*`.

Die Größe ist linear in $|\varphi|$.

($\Rightarrow$) Sei α eine erfüllende Belegung. Wähle für jedes j ein t_j, dessen Literal in C_j unter α wahr ist, und setze $c(K_j) = t_j.0.0$. Setze $c(X_i) = 2.0.0$, falls $\alpha(x_i) =$ wahr, sonst 1.0.0, für alle X_i, die von einer gewählten Version gefordert werden. Die Anforderungen der Wurzel sind erfüllt; die Anforderung der gewählten Version von K_j ist es, weil das Literal wahr ist: bei x_i ist $c(X_i) = 2.0.0 \geq 2.0.0$, bei $\neg x_i$ ist $c(X_i) = 1.0.0 < 2.0.0$. Die X_i haben keine Anforderungen. Also ist c zulässig.

($\Leftarrow$) Sei c zulässig. Setze $\alpha(x_i) =$ wahr genau dann, wenn X_i ausgewählt ist und $c(X_i) = 2.0.0$. Für jedes j ist K_j ausgewählt (Wurzel), etwa mit $c(K_j) = t.0.0$. Dessen Anforderung verlangt für das t-te Literal ℓ von C_j: X_i ist ausgewählt und $c(X_i) \geq 2.0.0$ (bei $\ell = x_i$), also $\alpha(x_i) =$ wahr und ℓ wahr; oder $c(X_i) < 2.0.0$ (bei $\ell = \neg x_i$), also

$\alpha(x_i) =$ falsch und ℓ wahr. Jede Klausel hat ein wahres Literal, also erfüllt α die Formel φ. □

Korollar 10.1 (Eindeutige Auflösung ist notwendig für Polynomialzeit)**.** Gäbe es einen Lader, der bei mehreren Kandidatendateien je Name stets eine zulässige Auswahl findet, wenn eine existiert, so entschiede er DepSelect; er läuft dann im ungünstigsten Fall nicht in polynomieller Zeit, sofern P $\neq$ NP.

Proof. Aus Satz 10.2 und der Definition der NP-Vollständigkeit. □

Satz 10.3 (Unvollständigkeit, Polynomialität des Laders)**.** Der Lader löst jeden Namen durch `locate` zu einer einzigen Datei auf (Satz 6.3).

(a) Es gibt eine Konfiguration, in der eine Datei, die einen Bereich erfüllt, im Suchpfad liegt, und `open` dennoch mit `DEPENDENCY_MISSING` scheitert.

(b) `open`(r) läuft in Zeit $O\big((1+|E_r|)\cdot n_{\max} + \sum_{f\in V_r}|f|\big)$ mit den Dateigrößen $|f|$ und den Kosten der Bereichsprüfungen und Hooks nicht gerechnet.

Proof. (a) Sei `SL_PATH`$= d_1$`:`d_2; `d1/base.sl` habe die Version 1.0.0, `d2/base.sl` die Version 2.0.0, und `top` fordere `base` mit `>=2.0.0`. `locate` liefert die Datei in d_1; die Bereichsprüfung scheitert und `_open_dependency` lässt den Fehler weiterlaufen. Die Datei in d_2 wird nie betrachtet. (b) Folgt aus Satz 7.1 und der Tatsache, dass `_read` und `unpack` jede Datei einmal in linearer Zeit lesen und zerlegen. □

Der Zusammenhang zu den übrigen Arbeiten der Reihe ist keine Zufälligkeit: Die Kette 3-SAT $\to$ Clique $\to$ Teilgraph der Vorarbeit und die Reduktion in Satz 10.2 sind dieselbe Art von Aussage über verschiedene Gegenstände. Dass der Entwurf bei widersprüchlichen Kandidaten einen Fehler verlangt, statt zu suchen, ist folglich auch aus Sicht der Berechenbarkeit gut begründet; die Frage, wie man die Diagnose bei Mehrdeutigkeit verbessert, ohne zu suchen, behandelt der Ausblick.

Kapitel 11

Sicherheit

11.1 Angreifermodell

Betrachtet werden drei Angreiferfähigkeiten: (A1) Der Angreifer kann Umgebungsvariablen des Prozesses setzen (nicht aber das Programm ändern). (A2) Er kann Dateien in Verzeichnissen anlegen, die für alle beschreibbar sind. (A3) Er kann Dateien zwischen zwei Zugriffen des Laders ersetzen, soweit seine Rechte es zulassen. Ziel ist, den Prozess zum Laden von Code zu bringen, den der Betreiber nicht vorgesehen hat.

11.2 Die klassischen Angriffswege

Angriffsweg	Maßnahme in `sl`	Bleibender Rest
Arbeitsverzeichnis (DLL-Hijacking)	Satz 6.2: nicht durchsucht, wenn R, A, Σ_{sys} absolut	R und A werden nicht normalisiert (B2)
`PATH`, `LD_LIBRARY_PATH`, `DYLD_LIBRARY_PATH`	Satz 6.1: nie gelesen	Windows liest `ProgramData`, `ProgramFiles` für Σ_{sys}
`SL_PATH` (A1)	Satz 6.4(i): im gehärteten Modus ignoriert	im Normalbetrieb vertraut der Lader der Variable
Beschreibbare Verzeichnisse (A2)	Satz 6.4(ii), (iii): Ablehnung, fail closed	nur unmittelbares Verzeichnis, nur *others*, Fehler bei `stat` lässt zu (B3)
Veränderter Payload	Satz 5.2: Hash	kein Schutz vor gezielter Veränderung (Bemerkung 5.1)
Nicht deklarierte Symbole	Satz 7.7	keine Sicherheitsgrenze im Prozess

Tabelle 11.1: Angriffswege, Gegenmaßnahmen und Rest.

11.3 Einmaliges Lesen

Lemma 11.1 (Validierte Bytes sind die abgebildeten Bytes)**.** Die Bytes, die `unpack` prüft, sind dieselben, aus denen der Payload geschnitten, auf Hash geprüft und in die temporäre Datei geschrieben wird.

Proof. `Loader._load` liest die Datei einmal (`self._read(path)`) und übergibt die Byte-Folge an `unpack`; `payload_for` schneidet aus ihrem Payload-Bereich; `map_payload` schreibt genau diese Folge. Es gibt keinen zweiten Lesezugriff auf den Pfad. □

Das Lemma schließt eine häufige Klasse von Wettläufen aus (Prüfen eines Pfades, späteres Öffnen eines anderen Inhalts). Offen bleibt der Wettlauf zwischen der Verzeichnisprüfung in `locate` und dem Lesen in `_read`, und das Öffnen der temporären Datei durch den Systemlader nach dem Schreiben. Unter POSIX ist das Verzeichnis nach der Dokumentation von `mkdtemp` nur für den Besitzer zugänglich; unter Windows hängt der Schutz von den Zugriffsrechten des übergeordneten Verzeichnisses ab **[nicht geprüft]**. Da der Ort über `TMPDIR`, `TEMP` oder `TMP` bestimmt wird, gehört auch diese Variable zum Vertrauensbereich des Prozesses (A1).

Kapitel 12

Anwendung auf die Beispielprojekte

12.1 Vorher und nachher

Der Entwurf zeigt die Wirkung an einem Beispiel (Abschnitt 12):

Listing 12.1: Vorher: plattformabhängiger Dateiname (nach `proposal.md`, Abschnitt 12).

```
def library_filename() -> str:
    if sys.platform == "win32":
        return "wsat_shim.dll"
    if sys.platform == "darwin":
        return "libwsat_shim.dylib"
    return "libwsat_shim.so"
```

Listing 12.2: Nachher: dieselbe Zeile auf jedem System.

```
import sl
lib = sl.open("wsat_shim")      # same code on every system
```

Tabelle 12.1 prüft, welche der sechs Stellen aus Abschnitt 2.2 das Paket `sl` *tatsächlich* ersetzt.

Stelle	Ersetzt?	Begründung
`library_filename()` mit vier Namen	ja	logischer Name, Lemma 6.1; Voraussetzung: die Bibliothek wird als `.sl` gebaut
`add_dll_directory` für `PATH`-Einträge	für deklarierte Abhängigkeiten ja	Abhängigkeiten werden nach derselben Ordnung aufgelöst (Satz 6.1); native Laufzeit-DLLs sind nicht erfasst
MinGW-Sonderfall in `build_shim()`	teilweise	`sl build` kennt nur GCC/Clang-Flags (Satz 9.3, B9)
Makro `CSUB_EXPORT`	ja	erzeugter Header mit `SL_EXPORT` (Listing 9.2)
statisches Linken der MinGW-Laufzeit	nein	das Feld `runtime` hat keine Wirkung (B11)
Plattformteile der README	teilweise	`sl build` ist überall gleich; Umgebungsvariablen und Shell bleiben verschieden

Tabelle 12.1: Ersatz der sechs Plattformstellen durch `sl` (Befunde in Kapitel 14).

Die Tabelle ist ernüchternder als der Entwurf: Von sechs Stellen entfallen zwei ganz, drei teilweise, eine gar nicht. Das ist ein Ergebnis dieser Arbeit und kein Einwand gegen den Entwurf: Es zeigt, welche Teile des Referenzpakets noch ausgebaut werden müssen, damit die Zusage des Entwurfs trägt (Ausblick, Abschnitte 16.7 und 16.12).

12.2 Metadaten für `wsat_shim`

Der Entwurf skizziert die Metadaten der Bibliothek in TOML (Anhang A); in der Implementierung sind sie ein JSON-Objekt. Die Entsprechung:

Listing 12.3: Metadatenskizze (`proposal.md`, Anhang A, gekürzt).

```
[sl]
container_version = 1
name         = "wsat_shim"
version      = "1.4.0"
abi_version = 3

[[target]]
os = "linux"
arch = "x86_64"
payload = "elf"

[runtime]
cpp = "embedded"

[[dependency]]
name = "cryptosolver"
version = ">=1.2,<2"

[exports]
symbols = ["wsat_abi_version", "wsat_solve", "wsat_free"]

[lifecycle]
init = "wsat_init"
fini = "wsat_fini"
```

Die Unterschiede zur Implementierung: `payload` (`elf`) ist dort nicht Teil der Metadaten, weil sich das Format aus dem Target ergibt; `cpp` heißt `runtime` mit den Werten `embedded` und `external`; Abhängigkeiten sind ein Wörterbuch Name → Bereich; der Bereich `>=1.2,<2` müsste `>=1.2.0,<2.0.0` heißen; `container_version` steht im Kopf und nicht in den Metadaten.

12.3 Ablauf eines `open` mit Abhängigkeit

Abbildung 12.1 zeigt den Ablauf von `sl.open("top")` für eine Bibliothek `top`, die `base` deklariert. Er entspricht dem Test `test_dependency_chain_and_release` (dort mit einer Zwischenbibliothek mehr).

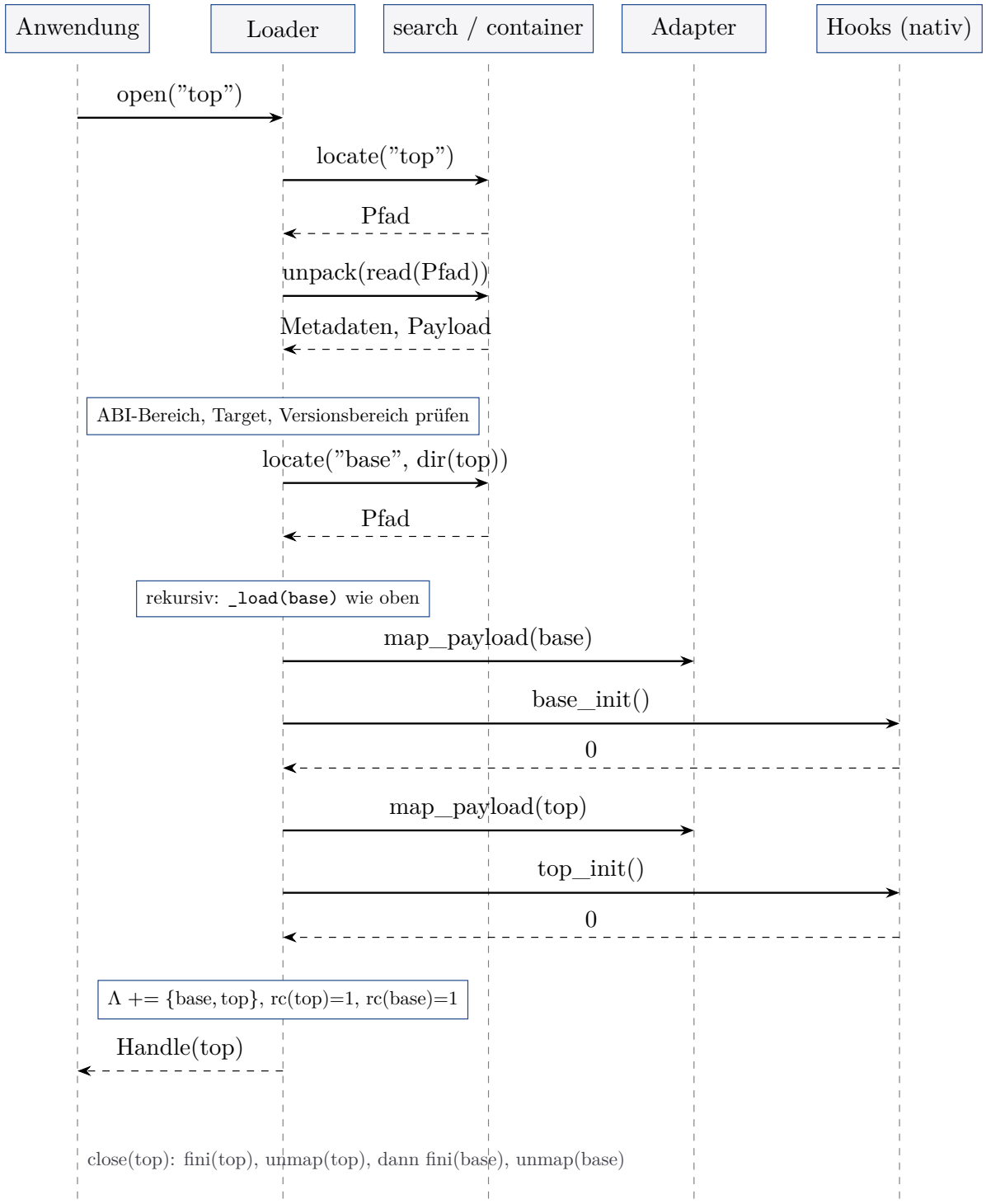


Abbildung 12.1: Ablauf von `open("top")` mit der Abhängigkeit `base`. Init-Reihenfolge nach Satz 7.4(i); die Freigabe läuft in umgekehrter Reihenfolge (Satz 7.4(iii)).

12.4 Der Test als Beleg: Reihenfolge der Abbildungen

Der Test `test_dependency_chain_and_release` (`tests/test_loader.py`) prüft für die Kette `top` → `mid` → `base` mit dem Fake-Adapter, dass `map` in der Reihenfolge `base`, `mid`, `top` und `unmap` in der Reihenfolge `top`, `mid`, `base` geschieht:

Listing 12.4: Ausschnitt aus `tests/test_loader.py` (Zeilen 92–100).

```
def test_dependency_chain_and_release(loader, adapter, write_sl):
    write_sl("base")
    write_sl("mid", dependencies={"base": ">=1.0.0"})
    write_sl("top", dependencies={"mid": "*"})
    handle = loader.open("top")
    assert adapter.mapped == ["base", "mid", "top"]       # Abhaengigkeiten
    ↪ zuerst
    assert loader.info(handle)["dependencies"] == ["mid"]
    loader.close(handle)
    assert adapter.unmapped == ["top", "mid", "base"]
```

Das ist ein Einzelfall der Aussagen (i) und (iii) von Satz 7.4; der Satz gilt für jeden azyklischen Graphen. Der Test belegt also eine Instanz, der Satz die Regel.

Teil IV

Prüfung

Kapitel 13

Berichtete Ergebnisse

Dieses Kapitel gibt wieder, was die README und der Abdeckungsbericht (`doc/coverage`, erzeugt am 21. Oktober 2026, 11:54 Uhr, Pfade mit Rückstrich, also auf Windows) berichten, und verbindet es mit den Sätzen dieser Arbeit. Es wurde nichts neu ausgeführt.

13.1 Umfang und Abdeckung

Das Paket umfasst 14 Module mit 1 170 Zeilen, die Tests 11 Dateien mit 1 150 Zeilen; die README nennt 126 bestandene Tests und eine Abdeckung von 99 %. Im Quelltext der Tests stehen 85 Testfunktionen, davon 10 parametrisiert; die Differenz zu 126 ist mit der Parametrisierung verträglich, wurde aber nicht nachgerechnet. Tabelle 13.1 gibt den Bericht wieder.

Modul	Anweisungen	fehlend	Zweige	teilweise	Abdeckung
`__init__`	9	0	0	0	100 %
`__main__`	3	0	0	0	100 %
`adapters`	65	0	0	0	100 %
`binding`	25	0	6	0	100 %
`build`	54	0	12	0	100 %
`capi`	41	0	2	0	100 %
`cli`	63	0	12	0	100 %
`codegen`	66	0	20	0	100 %
`container`	77	0	22	0	100 %
`errors`	27	0	0	0	100 %
`loader`	135	0	30	0	100 %
`metadata`	99	0	40	0	100 %
`platform_info`	21	0	6	0	100 %
`search`	95	4	50	4	96 %
Summe	780	4	200	4	99 %

Tabelle 13.1: Abdeckungsbericht (Berichtet). Zweigabdeckung ist eingeschaltet (`branch = true` in `pyproject.toml`).

Die vier fehlenden Anweisungen und vier teilweise abgedeckten Zweige liegen alle in

`search.py`: Zeile 21 (Rückgabe im Semikolon-Zweig), die Zeilen 125 bis 127 (POSIX-Zweig von `_world_writable`, auf Windows nicht erreicht), sowie die Zweige 20→21, 60→65, 62→65 und 120→125.

Lemma 13.1 (Ein toter Zweig in `split_sl_path`)**.** Im Listing 6.3 ist der Zweig „`if token` ist falsch“ am Ende der Funktion (Sprung 62→65) unerreichbar. Der Zweig „`if current` ist falsch“ (Sprung 60→65) ist erreichbar, etwa für `"/a:"`.

Proof. `current` wird nur durch `append` mit einem Ein-Zeichen-String (`raw[i]`, `":"` oder `"\\"`) gefüllt; ist die Liste nichtleer, ist `"".join(current)` ein nichtleerer String, also ist `token` wahr. Der Zweig 62→65 setzt dagegen ein falsches `token` bei wahrem `current` voraus. Für `"/a:"` wird beim letzten Doppelpunkt `current` auf die leere Liste gesetzt, die Schleife endet, und `if current` ist falsch. □

Der tote Zweig ist harmlos, aber auch nicht abdeckbar; ein Test kann ihn nicht erreichen. Die Prüfung `if token` im Schlussteil kann entfallen; das vereinfacht die Funktion und hebt die Zweigabdeckung der Datei.

13.2 Welche Sätze die Tests belegen

Tabelle 13.2 ordnet den Sätzen dieser Arbeit die Tests zu, die ihre Aussage an Beispielen prüfen, und nennt die Lücken. Ein Test ersetzt keinen Beweis, bindet ihn aber an den Quelltext: Ändert jemand das Verhalten, schlägt der Test an.

Tabelle 13.2: Beleg der Sätze durch Tests (Namen aus `tests/`).

Aussage	Test	Lücke
Satz 5.1 Rundreise	`test_fat_roundtrip`, `test_roundtrip_and_with_targets`	keine Zufallseingaben
Lemma 5.3 Prüfungen	`test_short_magic_version_and_truncation`, `test_unreadable_metadata`, `test_table_errors`	
Satz 5.2 Integrität	`test_hash_mismatch`	Metadaten ungeschützt (nicht geprüft)
Satz 5.3 Nichtkanonizität	–	kein Test
Lemma 6.1 Namen	`test_resolve_name`	
Satz 6.1 Struktur	`test_candidate_order_and_filters`	Windows-Systemverzeichnisse
Satz 6.2 Arbeitsverzeichnis	`test_cwd_is_never_searched`	relatives R
Satz 6.3 erster Treffer	`test_locate_hits_by_priority`	
Satz 6.4 gehärtet	`test_hardened_rejects_world_writable`, `test_requester_dir_and_hardened`	Gruppenrechte, Elternverzeichnisse
Satz 6.5 `SL_PATH`	`test_split_sl_path`	Semikolon-Zweig, Laufwerke

Aussage	Test	Lücke
Satz 7.2 Zyklen	`test_cycle_is_detected`	
Lemma 7.2(a) Rollback	`test_missing_dependency_rolls_back`, `test_init_failure_rolls_back`, `test_mapping_failure`	Nicht-`SLError`
Satz 7.3 Zählung	`test_refcount_shares_handle`, `test_shared_dependency_is_refcounted`, `test_closed_handle_is_rejected`	
Satz 7.4 Reihenfolge	`test_dependency_chain_and_release`, `test_init_runs_once_and_fini_on_close`, `test_missing_fini_does_not_block_unload`	
Satz 7.7 Export	`test_only_exported_symbols_are_visible`, `test_real_library_roundtrip`	
Satz 7.9 Deadlock	–	kein Test
Satz 9.1 Schichten	`test_generate_all`, `test_python_binding_is_valid_and_loadable`, `test_generated_python_binding_against_real_library`	
Lemma 9.2, Satz 9.2	`test_parse_signature`, `test_parse_signature_rejects`, `test_ctype_expr`	
Satz 9.3 Build	`test_build_sl_adds_host_to_extra_payloads`, `test_fat_container_and_cli_build`	
Satz 10.1 Bereiche	`test_satisfies`, `test_satisfies_rejects_bad_range`	unerfüllbare Bereiche

Drei Lücken fallen auf: Es gibt keinen Test für den Semikolon-Zweig von `SL_PATH` (der Bericht weist ihn als nie ausgeführt aus), keinen für Ausnahmen außerhalb von `SLError` in Hooks und keinen für nebenläufige Aufrufe. Alle drei sind in den Befunden 14.1, 14.5 und 14.4 wieder aufgenommen.

13.3 Einordnung der README-Aussagen

Die README sagt, geprüft worden sei unter Linux, und die Zeilenabdeckung habe bei 100 % gelegen. Der Bericht ist auf Windows erzeugt und weist für `search.py` 96 % aus (Tabelle 13.1). Die beiden Aussagen beschreiben vermutlich verschiedene Läufe; welcher gilt, lässt sich ohne Neulauf nicht sagen **[nicht geprüft]**. Die Unterschiede sind in Befund 14.12 festgehalten.

Kapitel 14

Befunde und Verbesserungsvorschläge

Dieses Kapitel fasst vierzehn Befunde zusammen. Jeder hat eine Kennung **B1** bis **B14**, einen Beleg (Satz oder Quelltextstelle) und einen Vorschlag. Befunde sind keine Fehler im strengen Sinn: Teils weichen sie vom Entwurf ab, teils benennen sie Annahmen, unter denen die Beweise gelten.

Befund 14.1 (B1: `SL_PATH` mit zwei Betriebsarten)**.** *Beleg:* Satz 6.5. Die Zerlegung wählt nach dem Inhalt (Semikolon vorhanden) die Maskierungsregeln; Einträge mit Semikolon sind nicht darstellbar; der Laufwerkszweig wirkt nur für den ersten Eintrag; der Semikolon-Zweig ist im Bericht ungetestet. *Vorschlag:* ein einziges Trennzeichen und eine Maskierung ohne Sonderfälle (Abschnitt 16.5), dazu Tests für beide Fälle und die Dokumentation in README und Entwurf.

Befund 14.2 (B2: Relative Verzeichnisse)**.** *Beleg:* Satz 6.2. `requester_dir` und `app_dir` werden nicht normalisiert; unter Windows können `ProgramData` und `ProgramFiles` relativ sein. Dann hängt die Suche vom Arbeitsverzeichnis ab. *Vorschlag:*

```
self.requester_dir = os.path.abspath(requester_dir) if requester_dir else None
self.app_dir = os.path.abspath(app_dir) if app_dir else None
# und im WindowsAdapter: nur absolute Systemverzeichnisse verwenden
```

Befund 14.3 (B3: Rechteprüfung im gehärteten Modus)**.** *Beleg:* Satz 6.4 und `search.py` Zeilen 113–127. Geprüft wird nur das Bit *others* des unmittelbaren Verzeichnisses; bei `OSError` wird zugelassen; unter Windows entscheidet der Verzeichnisname `open` (ein Testname) über das Ergebnis. *Vorschlag:* Besitzer und Rechte aller übergeordneten Verzeichnisse prüfen, bei Fehlern ablehnen, Windows-ACLs über eine eigene Funktion des Adapters abfragen, und den Testnamen aus dem Produktivcode entfernen (der Test kann sein Verzeichnis über einen Adapter-Stub kennzeichnen).

Befund 14.4 (B4: Globale Sperre über Hooks)**.** *Beleg:* Satz 7.9. Der Entwurf verlangt Initialisierung ohne eine Sperre, die Nutzercode betreten kann. *Vorschlag:* Sperren je Schlüssel mit Zuständen *Laden*/*Geladen* und Aufruf der Hooks außerhalb jeder Lader-Sperre (Abschnitt 16.3), dazu ein Test mit zwei Threads.

Befund 14.5 (B5: Ausnahmen außerhalb von `SLError`). *Beleg:* Satz 7.5(b). Unter Windows löst ein Speicherzugriffsfehler in nativem Code in `ctypes` eine `OSError` aus, die den Rollback umgeht. *Vorschlag:*

```
    except BaseException:
        if native is not None:
            self.adapter.unmap(native)
        for dep_handle in deps:
            self._release(dep_handle)
        raise
```

und im Fini-Pfad `unmap` und die Rückgabe der Abhängigkeiten in ein `finally`.

Befund 14.6 (B6: Native Abhängigkeiten). *Beleg:* Quelltext `adapters.py` Zeile 40, `ctypes.CDLL(path)` ohne Modus. Nach der Dokumentation von `ctypes` ist der Vorgabemodus unter POSIX *lokal*. Dann stellen die Metadaten-Abhängigkeiten die Reihenfolge von Init und Freigabe sicher, machen die Symbole der Abhängigkeit aber nicht für die abhängige Bibliothek auflösbar. Der Test `test_real_dependency_and_errors` deklariert eine Abhängigkeit, die der Code nicht nutzt **[Wirkung nicht ausgeführt geprüft]**. *Vorschlag:* Abbildungsmodus je Abhängigkeit in den Metadaten angeben (global/lokal) oder einen eigenen Abbilder einsetzen (Abschnitt 16.7), und einen Test, in dem die abhängige Bibliothek ein Symbol der Basis aufruft.

Befund 14.7 (B7: Container nicht kanonisch, ohne Echtheit). *Beleg:* Satz 5.3, Bemerkung 5.1. *Vorschlag:* kanonische Kodierung erzwingen (keine überzähligen Bytes, geordnete Tabelle, disjunkte Bereiche, kanonisches JSON) und eine Signatur über einen Digest von Metadaten, Tabelle und Payload-Hashes (Abschnitt 16.4).

Befund 14.8 (B8: Eingabeprüfung der Metadaten). *Beleg:* `metadata.py` Zeile 137: `tuple(data.get("exports", ()))` macht aus einer Zeichenkette `"wsat"` das Tupel `('w','s','a','t')`, das als Liste gültiger Ein-Zeichen-Bezeichner akzeptiert wird. Außerdem fängt `unpack` bei der Metadaten-Zerlegung nur `ValueError`; sehr tief verschachtelte JSON-Eingaben lösen in CPython eine `RecursionError` aus **[nicht ausgeführt]**. *Vorschlag:* Typprüfung (`isinstance(value, list)`) für Listenfelder und `except (ValueError, RecursionError)`.

Befund 14.9 (B9: Build-Werkzeug). *Beleg:* Satz 9.3 und die Folgetexte. GNU-Flags für alle Windows-Compiler, auch für `cl`; keine Angabe von Sprachstandard, Bibliotheken oder Linker; `cc` linkt keine C++-Laufzeit. *Vorschlag:* Optionen `--cc`, `--std`, `-l`, `--ldflag` und getrennte Toolchain-Beschreibungen für GNU und MSVC (Aufruf `cl /LD /Fe:`).

Befund 14.10 (B10: Erzeugtes CMake-Modul). *Beleg:* `codegen.py` Zeilen 96–110. Das Modul benutzt `SL_TARGET_PLATFORM`, `SL_TARGET_ARCH` und `Python3_EXECUTABLE`, ohne sie zu setzen, und stellt nur `C_VISIBILITY_PRESET` ein, obwohl die Zielprojekte C++ verwenden. *Vorschlag:* die Variablen aus `CMAKE_SYSTEM_NAME` und `CMAKE_SYSTEM_PROCESSOR` ableiten, `find_package(Python3)` aufrufen und `CXX_VISIBILITY_PRESET` und `VISIBILITY_INLINES_HIDDEN` ergänzen.

Befund 14.11 (B11: Felder und Daten ohne Wirkung). *Beleg:* Das Feld `runtime` wird gespeichert und validiert, aber vom Lader nirgends ausgewertet; der Parameter `flags` von `open` wird nicht benutzt; die erzeugte C-Quelle legt eine Tabelle `<name>_sl_`

`exports` an, die nirgends verwendet wird (bei `-Wall` kann das eine Warnung auslösen [**nicht geprüft**]); Lemma 13.1 nennt einen unerreichbaren Zweig. *Vorschlag:* die Felder mit Semantik belegen oder streichen; die Tabelle als Selbsttest verwenden (Vergleich mit den Metadaten) oder entfernen.

Befund 14.12 (B12: Dokumentationsstand)**.** *Beleg:* Kapitel 13. README: „Geprüft wurde unter Linux“, „Zeilenabdeckung 100 %“, Trennzeichen „`:` unter POSIX oder `;` unter Windows“; Bericht: Windows-Lauf, 96 % in `search.py`; Code: Betriebsart nach Inhalt. *Vorschlag:* Lauf und Bericht in CI erzeugen und die README-Angaben daraus ableiten.

Befund 14.13 (B13: Geteilte Handles)**.** *Beleg:* Satz 7.8 und die Bemerkung danach. Aufrufer teilen ein Handle-Objekt; ein doppeltes `close` nimmt die Referenz eines anderen zurück; `symbol` und `info` nehmen die Sperre nicht. *Vorschlag:* je `open` ein eigenes Zeugenobjekt (Token) herausgeben, das beim ersten `close` verbraucht wird, und `symbol` unter der Sperre des Schlüssels ausführen.

Befund 14.14 (B14: Kleinere Abweichungen vom Entwurf)**.** *Beleg:* (a) `lib`-Präfix im logischen Namen zulässig (Lemma 6.1); (b) die Liste der geprüften Kandidaten wird bei Erfolg verworfen (`_tried`); (c) `SLError.__str__` gibt nur Code und Meldung aus, nicht den mehrzeiligen Text des Entwurfs mit `failed_name` und Suchliste; (d) Versionsbereiche brauchen dreiteilige Versionen (`>=1.2` ist ungültig); (e) das Anwendungsverzeichnis hat keinen Vorgabewert; (f) Konstruktoren und `DllMain` laufen im Systemlader beim Abbilden, nicht umhüllt; (g) kein Legacy-Modus für `.dll`/`.so`/`.dylib`; (h) `sl pack` und `sl build` ersetzen das Targetfeld der Metadaten (Satz 9.3); (i) die Lader-API ist Python, nicht die C-Schnittstelle des Entwurfs; (j) die Abbildung über eine temporäre Datei setzt ein Verzeichnis mit Ausführungsrecht voraus. *Vorschlag:* die Punkte (a) bis (j) in die Spezifikation aufnehmen (als Abweichung begründen) oder umsetzen.

Die Befunde B1, B2, B5 und B8 sind Änderungen von wenigen Zeilen; B4, B6 und B7 verlangen einen Entwurf, der im Ausblick skizziert wird.

Teil V

Schluss

Kapitel 15

Zusammenfassung

Die Arbeit hat das Kapitel zum Laden der C/C++-Bibliothek aus der Arbeit zur Wiederverwendung des Subgraph Algorithmus zu einer eigenständigen, formalen Untersuchung des Vorschlags eines einheitlichen Shared-Library-Formats `.sl` ausgebaut. Gegenstand sind der Diskussionsentwurf und die Referenzimplementierung `sharedlib`, die gelesen, aber nicht ausgeführt wurde. Die Antworten auf die acht Fragen der Einleitung lauten:

1. **Plattformunterschiede.** Windows, Linux und macOS unterscheiden sich in Endung, Präfix, Binärformat, Lade-API, Standardsichtbarkeit, Abhängigkeitssuche, Initialisierung, Fehlermeldung und Entladen (Tabelle 2.1). In den zwei Beispielprojekten sind sechs Stellen reiner Plattformcode. Dass der Maschinencode selbst nicht vereinheitlicht werden kann, ist keine Vorsicht des Entwurfs, sondern ein Satz: Kein Register-Code-Block bedient zwei Aufrufkonventionen (Satz 2.1).

2. **Betriebssystemunabhängigkeit.** Sie lässt sich als Adapter-Parametrizität formulieren (Definition 4.1). Die Referenzimplementierung hat sie bis auf fünf benannte Ausnahmen (Satz 8.2, Tabelle 8.1): Pfadfunktionen, die Rechteprüfung, `realpath` und die Vorgabewerte. Die Konformitätsmatrix (Tabelle 4.1) bewertet 25 Forderungen des Entwurfs; die meisten sind erfüllt, wenige teilweise, eine nicht, und eine fehlt.

3. **Container.** Die Kodierung hat eine exakte Größenformel $18 + m + 80k + \sum |P(t)|$ (Lemma 5.1), eine bewiesene Rundreise (Satz 5.1) und prüft sieben Bedingungen (Lemma 5.3). Der Hash schützt Payloads nur gegen Zufall (Satz 5.2, Bemerkung 5.1); der Container ist nicht kanonisch (Satz 5.3).

4. **Suchordnung.** Die Kandidatenliste hat eine feste Struktur und hängt von der Umgebung nur über `SL_PATH` und, auf Windows, zwei Systemvariablen ab (Satz 6.1); sie ist unabhängig vom Arbeitsverzeichnis, wenn alle konfigurierten Verzeichnisse absolut sind (Satz 6.2); der erste Treffer gewinnt (Satz 6.3); der gehärtete Modus schließt `SL_PATH` aus und lehnt beschreibbare Verzeichnisse ab (Satz 6.4). Die Zerlegung von `SL_PATH` hat zwei vom Inhalt gewählte Betriebsarten (Satz 6.5).

5. **Lader.** Erfolg ist durch Zyklenfreiheit sowie lokale und Kantenbedingungen charakterisiert (Satz 7.2); Init und Freigabe laufen in Abhängigkeits- und

umgekehrter Reihenfolge (Satz 7.4); die Referenzzählung hat eine bewiesene Invariante (Satz 7.3); der Rollback stellt den Zustand wieder her, solange alle Ausnahmen `SLError` sind (Satz 7.5); die globale Sperre erlaubt einen Deadlock (Satz 7.9).

6. **Schichtung.** Der Importgraph ist azyklisch in sieben Ebenen, und der Laufzeitpfad schließt Codegenerierung, Build und Kommandozeile aus (Satz 8.1).

7. **Codegenerierung und Build.** Alle Schichten sind reine Funktionen der Metadaten (Satz 9.1); die Typabbildung ist unter einer Annahme über `ctypes` ABI-treu (Satz 9.2); das Build-Werkzeug ersetzt das Targetfeld (Satz 9.3) und deckt MSVC und C++ nicht ab.

8. **Versionen.** Versionsbereiche sind Intervalle (Satz 10.1); die Auswahl unter mehreren Kandidaten mit Rückverfolgung ist NP-vollständig (Satz 10.2); der Lader bleibt in Polynomialzeit, weil er den ersten Treffer nimmt, und ist deshalb unvollständig (Satz 10.3).

Hinzu kommen eine Sicherheitsanalyse mit dem Lemma vom einmaligen Lesen (Lemma 11.1), die Auswertung des Abdeckungsberichts mit einem bewiesenen toten Zweig (Lemma 13.1) und vierzehn Befunde. Ihr Kern: Der Entwurf trägt, die Referenzimplementierung setzt seine Zusagen für Container, Suche, Referenzzählung und Reihenfolgen nachweislich um; offen sind vor allem die Nebenläufigkeit (B4), die Ausnahmesicherheit (B5), native Abhängigkeiten (B6) und die Echtheit des Containers (B7). Für die Beispielprojekte folgt daraus ein ernüchterndes, aber klares Bild: Von den sechs Plattformstellen entfallen zwei ganz, drei teilweise, eine gar nicht (Tabelle 12.1).

Kapitel 16

Ausblick

Der Ausblick ist nach Gegenständen gegliedert: zuerst die Behebung der Befunde, dann die Bausteine, die der Entwurf verlangt und die Referenzimplementierung noch nicht hat, schließlich die Verbindung zu den übrigen Projekten, Messungen, Standardisierung und offene Forschungsfragen. Wo möglich, ist die Aussage so formuliert, dass sie sich beweisen oder messen lässt; bewiesen ist davon *nichts*, es sind Entwürfe und Beweisskizzen, und sie sind so gekennzeichnet.

16.1 Die Befunde beheben: eine Reihenfolge

Tabelle 16.1 ordnet die Befunde nach Aufwand und Wirkung. Die Reihenfolge ist so gewählt, dass jede Stufe die Aussagen der Sätze nicht verschlechtert und jede Behebung einen Test erhält, der die Lücke aus Tabelle 13.2 schließt.

Stufe	Befunde	Inhalt	Test
1	B2, B5, B8, B11	wenige Zeilen: Normalisierung, `BaseException`, Typprüfung, toter Code	Windows-Fall, Hook mit `OSError`, Container mit `exports` als Text
2	B1, B12, B14(a–e)	`SL_PATH`-Grammatik, Dokumentation aus CI, Diagnosetext	Round-Trip über beide Betriebsarten
3	B3, B9, B10	Rechteprüfung, Build-Optionen, CMake-Modul	Verzeichnisketten, Compiler-Matrix
4	B4, B13	Sperren je Schlüssel, Token-Handles	Zwei-Thread-Test mit blockierendem Init
5	B6, B7	native Abhängigkeiten, kanonischer Container mit Signatur	Aufruf in die Basis, Signaturvektoren

Tabelle 16.1: Vorschlag für die Reihenfolge der Behebung.

16.2 Aussagen als Eigenschaftstests und mechanisch geprüfte Beweise

16.2.1 Eigenschaftstests

Die Tests der Referenzimplementierung sind Beispieltests. Fast jeder Satz der Arbeit lässt sich als Eigenschaft über zufällig erzeugte Eingaben formulieren und mit einem Werkzeug im Stil von QuickCheck [8] prüfen:

- *Rundreise* (Satz 5.1): für zufällige gültige (M, P) gilt `unpack(pack(`M, P`))` $= (M, P)$.
- *Maskierung* (Satz 6.5(c)): für zufällige Listen nichtleerer, semikolonfreier Zeichenketten gilt $\mathrm{split}(\mathrm{join}(\cdot)) = \mathrm{id}$ unter der Randbedingung.
- *Zyklen und Erfolg* (Satz 7.2): für zufällige Abhängigkeitsgraphen auf ≤ 8 Knoten (aus Fake-Containern gebaut) gelingt `open` genau dann, wenn der erreichbare Teilgraph azyklisch und lokal gültig ist; das Orakel dazu ist ein Algorithmus nach Tarjan [6].
- *Bilanz* (Satz 7.3): für zufällige Folgen von `open`/`close` ist am Ende $\Lambda = \emptyset$ und jedes `map` durch ein `unmap` beantwortet.
- *Reihenfolge* (Satz 7.4): in jedem Ablauf eines zufälligen Graphen ist die Folge der Init-Ereignisse eine umgekehrte topologische Ordnung.
- *Rollback* (Lemma 7.2): für zufällig fehlschlagende Orakel ist nach einem Fehler der Zustand gleich dem vorherigen.
- *Struktur der Suche* (Satz 6.1): die Kandidatenliste ändert sich nicht, wenn `PATH`, `LD_LIBRARY_PATH` und `DYLD_LIBRARY_PATH` beliebig gesetzt werden.

Diese Tests sind klein, und sie fangen genau die Fehlerklasse, die Beispieltests übersehen: Randfälle der Verbindung von Eigenschaften. Der Test zum Beispiel 6.1 würde B1 sofort zeigen.

16.2.2 Mechanisch geprüfte Beweise

Das Modell aus Kapitel 3 ist klein genug für einen Beweisassistenten. Ein Vorgehen in drei Stufen:

1. *Zustandsmodell in TLA+* [7]: Zustand (Λ, σ), Aktionen `Open`, `Close`, Hooks, Orakelantworten als nichtdeterministische Wahl; Invarianten sind Satz 7.3 und Abgeschlossenheit; Eigenschaften der Lebendigkeit sind der Ausschluss von Deadlocks in der Variante mit Schlüssel-Sperren. Der Modellprüfer wird auf Graphen bis zu fünf Knoten mit zwei Threads angesetzt.
2. *Beweise in einem Beweisassistenten* (Lean oder Coq): Satz 7.2 und Lemma 7.2 als Programmeigenschaften eines funktionalen Modells von `_open`; Satz 10.2 als Reduktion mit Korrektheitsbeweis.

3. *Verbindung zum Quelltext:* Aus dem Python-Code wird ein funktionales Modell von Hand abgeleitet; Abweichungen zwischen Modell und Code sind die eigentliche Fehlerquelle, die Eigenschaftstests aus dem vorigen Abschnitt fangen.

Die Kosten sind erheblich; der Nutzen besteht vor allem darin, dass die Annahmen (3.1, 7.1, 9.1) sichtbar werden und jeder Satz nur diejenigen verwendet, die er braucht.

16.3 Ein Lader mit Sperren je Schlüssel

Befund B4 verlangt, die globale Sperre durch eine feinere Steuerung zu ersetzen. Der folgende Entwurf ist nicht implementiert.

16.3.1 Zustände und Regeln

Jeder Schlüssel k hat einen Zustand $z(k) \in \{\text{Leer}, \text{Lädt}(T), \text{Geladen}\}$ mit dem ladenden Thread T. Eine kurze Sperre $\mathcal{L}$ schützt nur die Tabelle z und die Zähler und wird *nie* über einen Hook, ein `dlopen` oder einen Dateizugriff gehalten. Die Regeln:

1. `open`(k) setzt unter $\mathcal{L}$ den Zustand von *Leer* auf *Lädt*(T) und lädt *außerhalb* von $\mathcal{L}$; bei *Geladen* wird der Zähler erhöht; bei *Lädt*(T') mit $T' \neq T$ wartet der Thread auf eine Bedingungsvariable zu k.

2. Bei *Lädt*(T) durch denselben Thread (Wiedereintritt aus einem Hook) wird `CYCLE` gemeldet.

3. Wartet T auf k, wird vorher geprüft, ob das Warten einen Kreis im Warte-Graphen der Threads schlösse (Thread T wartet auf T', der auf T'' wartet …auf T); in dem Fall wird `CYCLE` gemeldet.

4. Hooks laufen ohne jede Lader-Sperre.

Satz 16.1 (Beweisskizze: Verklemmungsfreiheit des Entwurfs)**.** Im Entwurf kann kein Deadlock auftreten, der durch den Lader entsteht: Jeder Kreis im Warte-Graphen wird entweder durch Regel 3 verhindert, oder er enthält ein Warten, das nicht vom Lader ausgeht (ein Hook wartet auf Nutzercode).

Skizze. Ein Thread wartet nur an Regel 1 (auf eine Bedingungsvariable zu k, die der ladende Thread T' nach Abschluss oder Fehler signalisiert) und niemals auf $\mathcal{L}$ länger als für eine kurze Tabellenoperation. Ein Warten von T auf T' entsteht nur, wenn T' einen Schlüssel lädt, den T für eine Abhängigkeit braucht. Ein Kreis $T_1 \to T_2 \to \cdots \to T_1$ im Warte-Graphen bedeutete, dass jeder T_i einen Schlüssel lädt, den T_{i-1} braucht; die Wartekanten wären also Abhängigkeitskanten, und Regel 3 erkennt den Kreis beim Eintragen der letzten Kante. Hooks halten keine Lader-Sperre; ein Thread, der in einem Hook auf Nutzercode wartet, wartet nicht auf den Lader. Ein Kreis, an dem solche Kanten beteiligt sind, geht vom Nutzercode aus und ist nicht Sache des Laders. Ein vollständiger Beweis müsste zusätzlich Ausnahmen (ein ladender Thread, der abbricht, muss $z(k)$ zurücksetzen und die Wartenden wecken) und das Verhalten bei mehrfachen Abhängigkeiten auf denselben Schlüssel behandeln. □

16.3.2 Auswirkungen auf die Sätze

Die Charakterisierung des Erfolgs (Satz 7.2) bleibt für jeden einzelnen Thread gültig; Lemma 7.2 wird zu einer Aussage über *linearisierbare* Operationen, die Invariante von Satz 7.3 wird unter $\mathcal{L}$ gehalten. Neu hinzu kommt eine Aussage über die Fairness des Wartens und eine Entscheidung, ob ein Thread beim Scheitern des ladenden Threads *selbst* nochmals lädt oder den Fehler übernimmt; der Entwurf sollte den Fehler jedem Wartenden melden, damit das Ergebnis nicht von der Thread-Reihenfolge abhängt.

16.4 Kanonischer Container und Signaturen

Befund B7 verlangt Echtheit. Der Entwurf von `proposal.md` nennt Hash und Signatur als Option (Abschnitt 10). Eine tragfähige Lösung hat drei Bausteine.

16.4.1 Kanonische Kodierung

Definition 16.1 (Kanonischer Container, Entwurf)**.** Ein Container heißt *kanonisch*, wenn (1) hinter dem Payload-Bereich keine Bytes stehen, (2) die Tabelle nach Targets aufsteigend sortiert und ohne Doppelungen ist, (3) die Payload-Bereiche in Tabellenreihenfolge lückenlos und überlappungsfrei liegen, (4) die Metadaten als kanonisches JSON (feste Schlüsselreihenfolge, keine überflüssigen Leerzeichen, Zahlen in Normalform, etwa nach RFC 8785 [10]) kodiert sind.

Satz 16.2 (Beweisskizze: Injektivität auf kanonischen Containern)**.** Auf der Menge der kanonischen Container ist `unpack` injektiv, und $\mathtt{pack} \circ \mathtt{unpack} = \mathrm{id}$.

Skizze. Aus (1) bis (3) folgt, dass die Längenangabe im Kopf, die Tabelle und die Offsets die Datei bis auf das letzte Byte bestimmen; (4) legt die Metadatenbytes fest. Die Kodierung durch `pack` erfüllt (1) bis (3) nach Lemma 5.1; sie erfüllt (4), wenn `json.dumps` durch die kanonische Kodierung ersetzt wird. Zwei kanonische Dateien mit gleichem Inhalt sind damit gleich. □

16.4.2 Signierter Digest

Gesucht ist ein Digest D, der Metadaten, Tabelle und Payload-Hashes bindet: $D = H(\text{Kopf} \,\|\, \text{Metadaten} \,\|\, \text{Tabelle})$ mit SHA-256 [9]; da die Tabelle die Hashes der Payloads enthält, bindet D alle Payloads. Die Signatur $\mathrm{Sig}_{sk}(D)$ mit einem Verfahren wie Ed25519 [11] wird in einen neuen Abschnitt hinter dem Payload-Bereich geschrieben, der von D *nicht* erfasst ist; der Kopf bekommt ein Feld für dessen Länge. Der Lader prüft die Signatur vor jeder Abbildung und gegen eine konfigurierte Menge vertrauter Schlüssel.

Satz 16.3 (Beweisskizze: Sicherheit)**.** Gibt es einen Angreifer, der eine akzeptierte Datei erzeugt, die nicht von einem Besitzer eines vertrauten Schlüssels stammt, so gibt es einen Angreifer gegen die Unfälschbarkeit des Signaturverfahrens oder gegen die Kollisionsresistenz von SHA-256.

Skizze. Eine akzeptierte Datei hat eine gültige Signatur über $D' = H(\ldots)$. Ist D' der Digest einer früher signierten Datei, so unterscheiden sich die Dateien in den gebundenen Teilen nicht (sonst Kollision) und in den Payloads nicht (die Hashes stehen in der Tabelle, wieder Kollision); sie sind bis auf ungebundene Teile gleich, die der Lader

nicht auswertet. Andernfalls hat der Angreifer eine Signatur für einen neuen Wert D' gefälscht. □

16.4.3 Weitere Fragen

Wie wird mit abgelaufenen oder zurückgezogenen Schlüsseln umgegangen (Sperrliste, Zeitstempel)? Wie wird die Signatur eines Fat-Containers behandelt, dem später ein Payload für ein neues Target hinzugefügt wird (Signatur über jeden Payload einzeln, mit Wurzel-Signatur)? Wie interagiert die Signatur mit reproduzierbaren Builds (gleiche Eingaben liefern gleiche Bytes)? Das sind Entwurfsfragen, die vor jeder Implementierung zu klären sind.

16.5 Eine Grammatik für `SL_PATH`

Befund B1 und die offene Frage des Entwurfs (Abschnitt 14) verlangen eine einzige Regel. Ein Vorschlag ist, *nur* das Semikolon als Trennzeichen zu verwenden und `\;` und `\\` als Maskierung. Der Doppelpunkt hat dann keine Sonderrolle, Laufwerksbuchstaben benötigen keinen Sonderfall, und Pfade mit Semikolon sind darstellbar.

Satz 16.4 (Beweisskizze: Unbedingte Rundreise)**.** Sei esc' die Maskierung von `\` und `;` und join' die Verbindung mit `;`. Für jede Liste nichtleerer Zeichenketten gilt $\mathrm{split}'(\mathrm{join}'(p_1, \ldots, p_k)) = (p_1, \ldots, p_k)$ ohne Randbedingung.

Skizze. Wie im Beweis von Satz 6.5(c), mit `;` statt `:` als Trennzeichen. Der Laufwerkszweig entfällt, und mit ihm die Randbedingung. Eingaben mit einem einzelnen `\` am Ende bleiben eine eigene Entscheidung: entweder Fehler oder wörtliches Zeichen; der Entwurf sollte sie festlegen und testen. □

Die Entscheidung hat einen Preis: Unter POSIX ist der Doppelpunkt der gewohnte Trenner; die Umstellung bricht Skripte, die `SL_PATH` wie `PATH` setzen. Als Übergang bietet sich eine Warnung an, wenn die Zeichenkette einen Doppelpunkt enthält und keinen Laufwerksbuchstaben. Alternativ erlaubt eine Listen-API (der Entwurf nennt sie) die Umgehung der Zeichenkette ganz.

16.6 Saubere Trennung zwischen Lader und Plattform

Tabelle 8.1 zeigt Plattformzugriffe außerhalb des Adapters. Sie lassen sich beseitigen, indem der Lader nicht mit Zeichenketten, sondern mit einer *Plattform* arbeitet, die von außen übergeben wird:

- einem Pfadtyp mit Operationen `join`, `parent`, `is_absolute`, der für POSIX und Windows rein (ohne Dateisystemzugriff) und plattformunabhängig implementiert ist (in Python: `PurePosixPath`, `PureWindowsPath`);
- einer Dateisystem-Schnittstelle `is_file`, `realpath`, `mode_of`, `owner_of`, die der Adapter implementiert und die in Tests durch ein Fake-Dateisystem ersetzt wird;
- den Vorgaben für Plattform und Architektur als Teil des Adapters.

Mit dieser Trennung wird Satz 8.2 zur Aussage ohne Ausnahmen: Die Beobachtung ist eine Funktion von (E, R, A, h) und dem Orakel einschließlich Dateisystem. Als Gewinn fiele der Test `test_hardened_rejects_world_writable` aus dem plattformabhängigen Teil heraus: Er liefe mit einem Fake-Dateisystem auf jedem System gleich, und der Testname-Sonderfall in `_world_writable` (B3) entfiele.

16.6.1 Ein Lader-Kern in C

Der Entwurf nennt als Stufe 1 einen kleinen C-Kern mit Plattformadaptern. Er hat zwei Folgen. Erstens erhielten Sprachbindungen (Rust, Java, Node) *denselben* Lader und nicht eine Neuimplementierung: Der Satz 8.2 gälte für den Kern, und jede Bindung wäre eine dünne Hülle. Zweitens entfiele die Abhängigkeit des Laders von der Python-Laufzeit; das Laden aus C++-Programmen (etwa `github.com/naphets86/csubgraph`) wäre direkt möglich. Der Preis ist ein zweiter Code, der gegen dieselbe Spezifikation geprüft werden muss; hierfür taugen sprachneutrale Prüfvektoren (Abschnitt 16.11).

16.6.2 Ein eigener Abbilder (Stufe 2)

Die Abbildung über eine temporäre Datei (Listing 8.1) hat drei Schwächen: Sie braucht ein Verzeichnis mit Ausführungsrecht; sie hinterlässt kurzzeitig eine Datei, deren Schutz vom Betriebssystem abhängt; und sie überlässt dem Systemlader die Auflösung der Symbole zwischen Bibliotheken. Alternativen:

- Anonyme Dateien im Speicher (unter Linux `memfd_create` mit `dlopen` über den Dateideskriptor); nicht portabel, aber in einem Linux-Adapter möglich.
- Ein Abbilder im Benutzerraum, der ELF, PE und Mach-O selbst in den Speicher bringt, Relokationen anwendet und Symbole selbst auflöst. Das macht das Verhalten unabhängig von den Eigenheiten der Systemlader und löst B6, ist aber ein großer Eingriff und vergrößert die Angriffsfläche, wie der Entwurf in Abschnitt 8.5 selbst einräumt.
- Betriebssystemunterstützung (Stufe 3 des Entwurfs): politisch und technisch am aufwendigsten.

16.7 Native Abhängigkeiten, Laufzeit und ABI-Entwicklung

16.7.1 Das Feld `runtime` mit Bedeutung füllen

Das Feld hat die Werte `embedded` und `external`, wird aber nicht ausgewertet (B11). Eine mögliche Bedeutung: Bei `embedded` bringt der Payload seine C++-Laufzeit statisch mit (wie die statisch gelinkte MinGW-Laufzeit in Gen-DB); der Build prüft das (etwa über die Linker-Optionen), und der Lader muss nichts tun. Bei `external` wird die Laufzeit als eigene `.sl`-Abhängigkeit deklariert (`libstdcxx` mit Version), und der Lader löst sie wie jede Abhängigkeit auf. Beide Fälle ersetzen die Plattformsonderfälle aus Abschnitt 2.2.

16.7.2 Symbolauflösung zwischen Bibliotheken

Für B6 sind drei Wege denkbar: der Modus `RTLD_GLOBAL` für Abhängigkeiten, die Symbole bereitstellen, bei Beibehaltung des lokalen Modus für Blätter; eine Angabe je Abhängigkeit in den Metadaten (`"binding": "global"`); oder der eigene Abbilder. Der Test dazu ist einfach: Eine Basis exportiert eine Funktion, eine zweite Bibliothek ruft sie auf, ohne sie selbst zu enthalten, und `sl.open` der zweiten muss gelingen.

16.7.3 Regeln der ABI-Entwicklung

Der Lader prüft bisher $\alpha \in [\text{low}, \text{high}]$ mit der Vorgabe $(1, 1)$, also Gleichheit. Für eine Praxis mit mehreren Versionen braucht es Regeln: Wann erhöht sich α? Gilt eine Bibliothek mit $\alpha = 3$ als verträglich mit Nutzern, die $\alpha = 2$ erwarten? Ein Vorschlag: α wird nur bei inkompatiblen Änderungen der exportierten C-Schnittstelle erhöht; Erweiterungen (neue Exporte) erhöhen nur die Minor-Version; jede Abhängigkeit trägt neben dem Versionsbereich einen ABI-Bereich. Die Bereichssprache (Satz 10.1) trägt das ohne Änderung, weil ABI-Bereiche Intervalle sind.

16.8 Signaturen, Besitz und weitere Sprachen

Die Signaturen der Metadaten sind C-Deklarationen und erlauben nur Skalare und Zeiger (Satz 9.2). Für wirkliche Bibliotheken fehlen mindestens: Strukturen (`ctypes.Structure`) mit Layout-Prüfung, Rückrufe (Funktionszeiger), Felder fester Länge, Aufzählungen, Eigentums- und Lebensdaueranmerkungen (*besitzt der Aufrufer den Rückgabezeiger, und mit welcher Funktion gibt er ihn frei?*), Fehlerrückgabe über Ausgabeparameter. Eine kleine Schnittstellenbeschreibung (IDL) in den Metadaten ersetzt dann die C-Deklaration als Quelle der Wahrheit; aus ihr erzeugt `sl gen` Header *und* Bindings für Python, Rust (`libloading`), Java (Panama) und Node-API. Die Aussage von Satz 9.1 (alle Schichten sind Funktionen der Metadaten) bleibt der Leitgedanke; sie muss für jede neue Sprache bewiesen oder getestet werden. Zu klären ist, wie das Datenmodell (LLP64/LP64) in der IDL ausgedrückt wird: Die Empfehlung ist, nur Typen fester Breite zuzulassen; dann entfiele der Datenmodellunterschied an der Schnittstelle ganz.

16.9 Build-System und Werkzeugkette

Nach B9 und B10 gehören zum Ausbau: Compiler- und Standardwahl, Linker-Angaben, Beschreibung von GNU- und MSVC-Toolchains als Daten (nicht als `if`-Zweige), ein belastbares CMake-Modul, Querübersetzung für die Zielmatrix, und ein Befehl `sl doctor`, der Compiler, Zielplattformen, Schreibrechte der Suchverzeichnisse und das Temporärverzeichnis prüft und das Ergebnis lesbar ausgibt. Die Build-Matrix (Satz 9.1) wird dann nicht nur berechnet, sondern in CI auf allen drei Systemen ausgeführt, mit je einer Zeile je Target, einer Zusage je Zeile und den Kosten pro Lauf. Reproduzierbare Builds sind Voraussetzung für Signaturen (Abschnitt 16.4).

16.10 Versionen: Diagnose statt Suche

Satz 10.2 schließt aus, dass der Lader unter Kandidaten sucht; er schließt *nicht* aus, dass er bei Mehrdeutigkeit hilft. Zwei Wege erhalten die Polynomialzeit:

1. *Diagnose.* Beim Scheitern an einem Bereich listet der Lader alle weiter hinten gefundenen Kandidaten mit Versionen auf (ein Durchlauf über die Kandidatenliste, $O(n_{\max})$ Dateikopf-Lesungen). Die Fehlermeldung nennt dann, dass `d2/base.sl` (Version 2.0.0) den Bereich erfüllt, aber hinter `d1/base.sl` liegt (Satz 10.3(a)).

2. *Sperrdatei.* Die Auswahl wird außerhalb des Laders getroffen (Paketverwaltung oder Build), und das Ergebnis, eine Abbildung Name → (Pfad, Hash), liegt als Sperrdatei vor. Der Lader *prüft* sie nur. Das passt zu Satz 10.2: Eine zulässige Auswahl ist ein Zeugnis in NP, und seine Prüfung ist polynomiell. Die Sperrdatei ist also genau das Zertifikat, das ein Lader in Polynomialzeit prüfen kann, auch wenn das Finden hart ist [12, 13].

Zu den Metadaten gehört außerdem die Prüfung, dass Bereiche erfüllbar sind (Satz 10.1(d)); sie ist billig, weil die Erfüllbarkeit einer Bereichsliste durch Vergleich der größten unteren mit der kleinsten oberen Schranke entschieden wird.

16.11 Konformitätssuite mit sprachneutralen Vektoren

Der Entwurf verlangt eine Konformitätssuite (Abschnitt 5.9), die jeder Lader, Adapter und jede Bindung bestehen muss. Die Tests der Referenzimplementierung sind dafür nicht geeignet, weil sie Python-Aufrufe prüfen. Sprachneutral sind *Prüfvektoren*: Eine Datei beschreibt ein Dateisystem (Pfade, Inhalte, Rechte), eine Umgebung, eine Folge von Operationen und die erwartete Beobachtung (Statuscodes, `failed_name`, `searched`, Reihenfolge der Hook-Aufrufe). Die Beobachtung ist nach Definition 4.1 genau das, worauf der Entwurf Wert legt. Aus dem Python-Modell können die Vektoren erzeugt werden, einschließlich der Zufallsfälle aus Abschnitt 16.2; ein Lader in C oder Rust wird gegen sie geprüft, ohne dass der Python-Code dafür vorhanden sein muss. Die Vektoren zu Satz 6.5 (Zerlegung von `SL_PATH`) und Satz 6.1 (Kandidatenliste) sind besonders einfach, weil sie reine Funktionen prüfen.

16.12 Migration der Beispielprojekte

Die Wirkung von `.sl` auf Gen-DB und `subgraph-sat-solver-econ` ist bislang eine Behauptung des Entwurfs. Ein Plan zum Prüfen:

1. *Metadaten schreiben.* Für die C-Schnittstelle von `csubgraph` (Shim für Gen-DB) und für `wsat_shim` je eine Metadatendatei mit Name, Version, ABI-Version, Targets, Exporten, Signaturen und Hooks. Die Signaturen sind vorhanden (als C-Header); sie müssen in das Fragment aus Definition 9.1 passen. Der Quelltext dieser Projekte lag der Arbeit nicht vor.

2. *Bau mit* `sl build`. Voraussetzungen sind die Lücken aus B9 (C++17, Linker, `cl`).

3. *Laden mit* `sl.open` anstelle von `ctypes.CDLL`; die alte Pfadlogik bleibt zunächst als Rückfall.

4. *Messung*: Wie viele Zeilen Plattformcode entfallen (Zählung vor und nach), wie viele Fehlerfälle werden eindeutig gemeldet, welche Zeit kostet das Laden, wie ändert sich die README (Abschnitte, die entfallen).

5. *Entscheidung nach Messung*, wie der Entwurf selbst es in seinem Fahrplan vorsieht (Abschnitt 13).

Als Hypothesen sind vorab festzulegen: **H1:** Die Zahl der Plattformverzweigungen in den Ladepfaden beider Projekte sinkt auf null. **H2:** Die Ladezeit steigt um weniger als eine feste Schranke (etwa 50 ms je Bibliothek für Payloads unter 10 MB). **H3:** Jeder in Abschnitt 2.2 genannte Fehlerfall liefert einen Statuscode des Fehlermodells statt einer Systemmeldung. Jede Hypothese ist falsifizierbar; **[offen]** bis zur Messung.

16.13 Kostenmodell und Messungen

Die Kosten von `open` setzen sich zusammen aus: Suchen ($\leq n_{\max}$ Dateisystemprüfungen), Lesen der Datei (linear in der Dateigröße S), Entpacken (der Code kopiert den Rest der Datei in `blob`, weitere $O(S)$), Hashen des Payloads (linear), Schreiben der temporären Datei (linear) und `dlopen`. Als Modell

$$T_{\text{open}}(S) \approx a + b\,S, \qquad b = \frac{1}{v_{\text{Lesen}}} + \frac{1}{v_{\text{Kopie}}} + \frac{1}{v_{\text{SHA}}} + \frac{1}{v_{\text{Schreiben}}},$$

mit einer Konstante a für Suche und `dlopen`. Zu messen sind a und b je System, mit kalten und warmen Zwischenspeichern, und der Vergleich mit direktem `ctypes.CDLL`. Bei den hier betrachteten Anwendungen (wenige Aufrufe mit großen Datenmengen, Entwurf Abschnitt 11) wird der Lader-Aufwand voraussichtlich klein gegen die Rechenzeit sein; ob das stimmt, ist **[offen]**. Verbesserungen, die das Modell nahelegt: Lesen mit Speicherabbildung statt Kopie, Hash nur beim ersten Laden und zwischengespeichert nach Dateistempel, Wiederverwendung der materialisierten Datei (Inhaltsadresse) statt jedes Mal neu zu schreiben, was zugleich die Temporärverzeichnisse spart.

16.14 Verbindung zum Subgraph-Programm

Für die Reihe der Arbeiten zum Subgraph Algorithmus ergeben sich drei Anschlüsse.

1. *Katalog.* Der Katalog der Wiederverwendungen (**W1** bis **W18**) in `csubgraph.tex` sollte um einen Eintrag der Form *Infrastruktur* ergänzt werden: das `.sl`-Format als Hülle der Bibliothek (Abschnitt 2.4). Die Zuordnung zu den Formen wäre unverändert.

2. *Orakel als Bibliothek.* Der Subgraph-SAT-Solver ruft die Subgraph-Bibliothek als Orakel (Vorarbeit). Ein `.sl`-Container mit einem Export, der ein Graphenpaar

entgegennimmt und eine Einbettung zurückgibt, wäre eine Schnittstelle, hinter der die Python-Referenz, die C++-Bibliothek und künftige Implementierungen austauschbar sind. Satz 9.1 beschreibt, wie die Signatur dafür zur einzigen Quelle würde.

3. *Beweisstil.* Die Kette 3-SAT → Clique → Teilgraph und die Reduktion in Satz 10.2 sind verwandt; die Reduktion auf die Versionsauswahl könnte in der Reihe als Beispiel dafür dienen, dass die Schwere eines Problems die Gestalt der Schnittstelle begründet (hier: der Lader wählt nicht).

16.15 Standardisierung und Betrieb

Der Entwurf stellt die Frage, wer Spezifikation, Referenzimplementierung und Konformitätssuite pflegt (Abschnitt 14). Ein tragfähiger Weg ist, die Spezifikation als versioniertes Dokument mit nummerierten Anforderungen zu führen, wie es Tabelle 4.1 vorbereitet, jede Anforderung an mindestens einen Prüfvektor zu binden und die Abweichungen der Referenzimplementierung (B14) offen zu führen. Das Risiko der Zersplitterung (Entwurf Abschnitt 11, *xkcd 927*) wird durch den Legacy-Modus gemildert, der gewöhnliche `.dll`-, `.so`- und `.dylib`-Dateien über dieselbe API öffnet; er fehlt bisher (B14(g)) und ist für die Übernahme in bestehende Projekte vermutlich wichtiger als jede andere Erweiterung. Zur Frage der Lizenz und des Betriebs: Das Repository enthält eine `LICENSE`-Datei; für eine Normierung wäre eine Lizenz zu wählen, die Übernahme in Systembibliotheken nicht verbietet.

16.16 Weitere Entwürfe des Entwurfs

Der Entwurf nennt Fragen, die diese Arbeit nicht beantwortet, aber präzisiert:

- *Mehrere Architekturen in einer Datei* (x86-64, ARM64): bereits durch den Fat-Container gelöst (Target = Plattform + Architektur); offen ist die Auswahl zwischen verträglichen Targets (etwa `x86` auf `x86_64`) und die Rolle von `musl` gegenüber `glibc`, die das Target nicht ausdrückt.
- *Verzögertes Binden und Laden:* Welche Semantik gilt für Symbole, die erst bei Gebrauch aufgelöst werden? Im Modell wäre ein dritter Zustand je Symbol nötig; Satz 7.7 bliebe wahr, aber Fehler träten später auf.
- *Entladen von Bibliotheken mit Threads oder Rückrufen:* Bemerkung zu `dlclose` in Kapitel 8; eine Zusage braucht einen Mechanismus, der Threads der Bibliothek beendet (ein `fini`-Vertrag).
- *Einbettung des Laders in Prozesse ohne Ausführungsrecht im Temporärverzeichnis:* Hinweis zur Stufe 2.

16.17 Offene Forschungsfragen

1. Lässt sich die Menge {erfolgreiche Abläufe} des Laders mit Sperren je Schlüssel so charakterisieren, dass Satz 7.2 unverändert für jeden Thread gilt, und wie ist die Fairness des Wartens zu fassen?

2. Gibt es eine *minimale* Spezifikation der Ladeprozedur, aus der sich alle Sätze dieser Arbeit ableiten lassen, und welche Annahmen sind darin unvermeidbar?

3. Welche Sicherheitseigenschaft gilt für den gehärteten Modus gegen den Angreifer (A3) formal, wenn Besitzer und Rechte aller übergeordneten Verzeichnisse geprüft werden, und was bleibt als nicht entscheidbarer Rest (etwa Zugriffsrechte, die sich zwischen Prüfung und Lesen ändern)?

4. Ist DepSelect auch NP-vollständig, wenn jede Version höchstens eine Anforderung hat und jedes Paket nur zwei Versionen? (Die Reduktion in Satz 10.2 braucht drei Versionen für Klauselpakete; für zwei ist die Frage offen und hängt mit 2-SAT zusammen.)

5. Welche Klasse von Signaturen lässt sich durch ein *beweisbar* ABI-treues Binding erfassen, und wie viel von der C-ABI der drei Plattformen muss das Modell kennen?

6. Wie groß ist der Anteil des Plattformcodes in Projekten, die C/C++-Bibliotheken aus Python nutzen, tatsächlich? Der Entwurf behauptet “tausendfach wiederholt”; eine Stichprobe öffentlicher Projekte würde die Größenordnung belegen oder widerlegen.

7. Wie verhält sich das Format zu WebAssembly mit Komponentenmodell, das ein Format und mehrere Laufzeiten verspricht: Welche Eigenschaften des Entwurfs (Export, Fehlermodell, Lebenszyklus) lassen sich dort ausdrücken, und was kostet die Schnittstelle?

Literaturverzeichnis

[1] S. Epp: *Subgraph Algorithmus – Wiederverwendung. Von der Python-Referenz über csubgraph zu Gen-DB, Subgraph-SAT-Solver und Experimenten.* Wissenschaftliche Arbeit (`csubgraph.tex`), 10. Oktober 2026.

[2] R. M. Karp: Reducibility among combinatorial problems. In: R. E. Miller, J. W. Thatcher (Hrsg.): *Complexity of Computer Computations*, Plenum Press, 1972, S. 85–103.

[3] M. R. Garey, D. S. Johnson: *Computers and Intractability: A Guide to the Theory of NP-Completeness.* W. H. Freeman, 1979.

[4] J. R. Levine: *Linkers and Loaders.* Morgan Kaufmann, 1999.

[5] U. Drepper: *How To Write Shared Libraries.* 2011.

[6] R. E. Tarjan: Depth-first search and linear graph algorithms. *SIAM Journal on Computing* 1(2), 1972, S. 146–160.

[7] L. Lamport: *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers.* Addison-Wesley, 2002.

[8] K. Claessen, J. Hughes: QuickCheck: a lightweight tool for random testing of Haskell programs. *Proc. ICFP*, 2000.

[9] National Institute of Standards and Technology: *FIPS PUB 180-4: Secure Hash Standard (SHS).* 2015.

[10] A. Rundgren, B. Jordan, S. Erdtman: *RFC 8785: JSON Canonicalization Scheme (JCS).* 2020.

[11] S. Josefsson, I. Liusvaara: *RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA).* 2017.

[12] R. Cox: *Version SAT.* Blogbeitrag, 2016 (research.swtch.com/version-sat).

[13] P. Abate, R. Di Cosmo, R. Treinen, S. Zacchiroli: Dependency solving: a separate concern in component evolution management. *Journal of Systems and Software* 85(10), 2012.